

ISTQB® Certified Tester

Certified Tester AI-Testing Syllabus

Versión 2.0

International Software Testing Qualifications Board

Proporcionado por Alliance for Qualification, Artificial Intelligence United, Chinese Software Testing Qualifications Board y Korean Software Testing Qualifications Board

Aviso de Derechos de Autor

Copyright Notice © International Software Testing Qualifications Board (en adelante ISTQB®)

ISTQB® es una marca registrada del International Software Testing Qualifications Board.

Copyright © 2026, los autores Klaudia Dussa-Zieger (presidenta), Stuart Reid, Vipul Kocher, Qin Liu, Jarosław Hryszko, Kyle Alexander Siemens y Werner Henschelchen.

Copyright © 2021, los autores Klaudia Dussa-Zieger (presidenta), Vipul Kocher, Qin Liu, Stuart Reid, Kyle Alexander Siemens, Werner Henschelchen y Adam Leon Smith.

Todos los derechos reservados. Los autores transfieren por la presente los derechos de autor al ISTQB®. Los autores (como titulares actuales de los derechos de autor) y el

ISTQB® (como futuro titular de los derechos de autor) han acordado las siguientes condiciones de uso:

- Se pueden copiar extractos, para uso no comercial, de este documento si se reconoce la fuente.
Cualquier Proveedor de Formación Acreditado puede utilizar este syllabus como base para un curso de formación si se reconoce a los autores y al ISTQB® como fuente y propietarios de los derechos de autor del syllabus, y siempre que cualquier publicidad de dicho curso de formación solo pueda mencionar el syllabus después de que la Acreditación oficial de los materiales de formación haya sido recibida de un Consejo Miembro reconocido por el ISTQB®.
- Cualquier individuo o grupo de individuos puede utilizar este syllabus como base para artículos y libros, si se reconoce a los autores y al ISTQB® como fuente y propietarios de los derechos de autor del syllabus.
- Cualquier otro uso de este syllabus está prohibido sin obtener primero la aprobación por escrito del ISTQB®.
- Cualquier Consejo Miembro reconocido por el ISTQB® puede traducir este syllabus, siempre que reproduzca el Aviso de Derechos de Autor mencionado anteriormente en la versión traducida del syllabus.

Historial de Revisiones

Versión	Fecha	Observaciones
1.0	2021/10/01	Publicación para GA
2.0 Alpha	2025/08/15	Revisión Alpha
2.0 Beta	2026/01/07	Revisión Beta
2.0	2026/04/17	Publicación para GA

Tabla de Contenidos

Agradecimientos

La Asamblea General del ISTQB® publicó formalmente este documento el 17 de abril de 2026.

0 Introducción

0.1 Propósito de este Syllabus

Este syllabus constituye la base para el ISTQB® Certified Tester AI Testing. El ISTQB® proporciona este syllabus de la siguiente manera: - A los consejos miembros, para su traducción al idioma local y para la acreditación de proveedores de formación. Los consejos miembros pueden adaptar el syllabus a las necesidades específicas de su idioma y modificar las referencias para alinearlas con sus publicaciones locales. - A los organismos de certificación, para elaborar preguntas de examen en su idioma local, adaptadas a los objetivos de aprendizaje de este syllabus. - A los proveedores de formación, para producir material didáctico y determinar los métodos de enseñanza apropiados. - A los candidatos a la certificación, para prepararse para el examen de certificación (ya sea como parte de un curso de formación o de forma independiente). - A la comunidad internacional de ingeniería de software y sistemas, para avanzar en la profesión de pruebas de software y sistemas, y como base para libros y artículos.

0.2 El Certified Tester AI Testing

El Certified Tester AI Testing (CT-AI) está diseñado para personas involucradas en las pruebas de sistemas basados en AI. Esto incluye personas en diversos roles, como testers, analistas de pruebas, analistas de datos, ingenieros de pruebas, consultores de pruebas, gestores de pruebas, testers de aceptación de usuario y desarrolladores de

software. Esta certificación también es adecuada para personas que buscan una comprensión fundamental de las pruebas de sistemas basados en AI, incluyendo gestores de proyecto, gestores de calidad, gestores de desarrollo de software, analistas de negocio, miembros de equipos de operaciones, directores de TI y consultores de gestión.

0.3 Trayectoria Profesional para Testers

El esquema del ISTQB® proporciona apoyo a los profesionales de pruebas en todas las etapas de sus carreras. Las personas que obtienen la certificación ISTQB® Certified Tester Foundation Level también pueden estar interesadas en los Niveles Avanzados Básicos (Test Analyst, Technical Test Analyst y Test Manager) y, posteriormente, en el Nivel Experto (Test Management o Improving the Test Process). La línea de Especialista ofrece una inmersión profunda en enfoques y actividades de prueba específicos, por ejemplo, Agile Testing, Test Automation, AI Testing, Testing with Generative AI o Mobile App Testing, o en conocimientos de pruebas grupales para determinados dominios industriales, por ejemplo, Automotive o Gaming. Visite www.istqb.org para obtener la información más reciente sobre el esquema Certified Tester del ISTQB.

0.4 Resultados de Negocio

Esta sección enumera los Resultados de Negocio esperados de un candidato que haya obtenido la certificación CT-AI.

Un Certified Tester en AI Testing puede: BO1 Comprender el estado actual de la AI, incluyendo la AI generativa. BO2 Experimentar la implementación y las pruebas de modelos de machine learning. BO3 Comprender el funcionamiento y las pruebas de redes neuronales simples. BO4 Comprender las características de calidad específicas de AI definidas por ISO/IEC 25059. BO5 Calcular e interpretar métricas de rendimiento funcional de ML para modelos de machine learning. BO6 Reconocer el alcance y la importancia de los dos niveles de prueba específicos para las pruebas de sistemas de machine learning. BO7 Contribuir al desarrollo de una estrategia de pruebas efectiva para un sistema de

machine learning. BO8 Diseñar y ejecutar casos de prueba para sistemas de machine learning.

0.5 Objetivos de Aprendizaje, Objetivos Prácticos y Nivel Cognitivo de Conocimiento

Los Objetivos de Aprendizaje (LO) respaldan los resultados de negocio y se utilizan para crear los exámenes CT-AI. Los niveles específicos de los objetivos de aprendizaje se muestran al inicio de cada capítulo y se clasifican de la siguiente manera: - K1: Recordar - K2: Comprender - K3: Aplicar - K4: Analizar

Se proporcionan más detalles y ejemplos de objetivos de aprendizaje en el Apéndice A.

Para todos los términos enumerados como palabras clave justo debajo de los encabezados de los capítulos, se deberá recordar (K1) el nombre correcto y la definición del Glosario del ISTQB® o del Capítulo 9, incluso si no se mencionan explícitamente en el objetivo de aprendizaje.

Los Objetivos Prácticos (HO) se centran en la aplicación práctica de los Objetivos de Aprendizaje y se muestran al inicio de cada capítulo. El nivel de un HO se clasifica de la siguiente manera: - H0: Puede incluir una demostración en vivo de un ejercicio o un vídeo grabado. Dado que el alumno no lo realiza, no es estrictamente un ejercicio. - H1: Ejercicio guiado. Los alumnos siguen una secuencia de pasos realizados por el formador. - H2: Ejercicio con pistas. Al alumno se le proporciona un ejercicio con pistas relevantes para permitir que el ejercicio se resuelva dentro del plazo establecido.

0.6 El Examen del Certificado Certified Tester AI Testing

El examen CT-AI se basará en los Objetivos de Aprendizaje descritos en este syllabus. Todas las secciones del syllabus son evaluables excepto la Introducción, los Objetivos Prácticos, las Referencias y los Apéndices. Responder a las preguntas del examen puede requerir el uso de material basado en más de una sección de este syllabus. Los

estándares y libros se incluyen como referencias, pero su contenido no es evaluable más allá de lo resumido en el propio syllabus.

Consulte el documento de Estructuras y Reglas de Examen para el CT-AI v2.0 para más detalles.

El criterio de entrada principal para cualquier persona interesada en realizar el examen CT-AI es poseer el certificado ISTQB® Certified Tester Foundation Level [CTFL].

Se recomienda encarecidamente que los candidatos también: - Tengan una experiencia mínima en desarrollo de software o en pruebas de software, como seis meses de experiencia como tester de sistema o de aceptación de usuario, científico de datos o desarrollador de software. - Realicen un curso acreditado según los estándares del ISTQB® (por uno de los consejos miembros reconocidos por el ISTQB).

0.7 Acreditación

Un Consejo Miembro del ISTQB® puede acreditar a proveedores de formación cuyo material de curso siga este syllabus. Los proveedores de formación deben obtener las directrices de acreditación del Consejo Miembro o del organismo que realiza la acreditación. Un curso acreditado se reconoce como conforme a este syllabus y permite que un examen del ISTQB® se incluya en el curso.

Las directrices de acreditación para este syllabus siguen las Directrices Generales de Acreditación publicadas por el Processes Management and Compliance Working Group.

0.8 Tratamiento de Estándares

Las organizaciones internacionales de estandarización como IEEE e ISO han emitido estándares asociados con las características de calidad y las pruebas de software. El propósito de estas referencias es proporcionar un marco (como en las referencias a ISO/IEC 25059 e ISO/IEC 25010 respecto a un modelo de calidad para sistemas basados en AI) o proporcionar una fuente de información adicional si el

lector lo desea. Tenga en cuenta que los syllabus utilizan los documentos de estándares como referencia. Los documentos de estándares no están destinados a ser evaluados en el examen.

0.9 Nivel de Detalle

El nivel de detalle en este syllabus permite cursos y exámenes internacionalmente consistentes. Para lograr este objetivo, el syllabus consta de: - Objetivos instructivos generales que describen la intención del syllabus CT-AI - Una lista de palabras clave que los estudiantes deben ser capaces de recordar - Objetivos de aprendizaje para cada área de conocimiento, que describen el resultado de aprendizaje cognitivo a alcanzar - Una descripción de los conceptos clave, incluyendo referencias a fuentes como literatura aceptada o estándares

El contenido del syllabus no describe toda el área de conocimiento de las pruebas de software; refleja el nivel de detalle que debe cubrirse en los cursos de formación CT-AI. Se centra en introducir los conceptos básicos de la AI y del machine learning (ML) en particular, y cómo pueden probarse los sistemas basados en estas tecnologías.

El syllabus utiliza la terminología (es decir, el nombre y significado) de los términos utilizados en las pruebas de software y el aseguramiento de la calidad según el Glosario del ISTQB®.

0.10 Cómo está Organizado este Syllabus

Hay siete capítulos con contenido evaluable. El encabezado de nivel superior de cada capítulo especifica la duración del capítulo; no se proporciona una temporización a nivel de capítulo. Para los cursos de formación acreditados, el syllabus requiere un mínimo de 19,5 horas de instrucción, distribuidas en los siete capítulos de la siguiente manera:

- Capítulo 1: 120 minutos – Introducción a la Inteligencia Artificial (AI)
- Comprender las diferencias clave entre los sistemas basados en AI y los sistemas convencionales, y

explorar el espectro de capacidades de la AI, que abarca desde la AI estrecha hasta la super AI.

- Obtener una comprensión fundamental de las tecnologías de AI, incluyendo la AI generativa (GenAI), y el hardware, el alojamiento y los marcos de desarrollo utilizados para implementar sistemas de machine learning (MLS).
- Aprender cómo las regulaciones y los estándares influyen en el desarrollo y las pruebas de soluciones basadas en AI.
- Capítulo 2: 45 minutos – Características de Calidad para Sistemas Basados en AI
 - Aprender sobre las características de calidad específicas de los sistemas basados en AI, incluyendo las definidas en ISO/IEC 25059, y comprender las consideraciones de seguridad al utilizar AI en sistemas críticos.
 - Explorar cómo definir criterios de aceptación apropiados adaptados al comportamiento y rendimiento únicos de las soluciones basadas en AI.
- Capítulo 3: 375 minutos – Machine Learning
 - Comprender los tipos de ML, los pasos clave en el flujo de trabajo de desarrollo de ML, y cómo los modelos preentrenados, el ajuste fino y la generación aumentada por recuperación contribuyen a los sistemas modernos basados en AI.
 - Aprender sobre la preparación de datos, los roles de los conjuntos de datos de entrenamiento, validación y prueba, y cómo estos influyen en el desarrollo y rendimiento del modelo de ML.
 - Explorar las redes neuronales, incluyendo su estructura y medidas de cobertura, y obtener experiencia práctica con métricas de rendimiento y el uso de una matriz de confusión.
- Capítulo 4: 195 minutos – Pruebas de Sistemas Basados en AI

- Comprender los desafíos únicos de las pruebas de sistemas basados en AI, incluyendo las diferencias en la testabilidad entre sistemas bloqueados y adaptativos, la necesidad de pruebas estadísticas y las dificultades de definir oráculos de prueba.
- Aprender a probar GenAI y modelos de lenguaje de gran escala (LLM), utilizando técnicas como red teaming y pruebas exploratorias para AI que realizan tareas de prueba.
- Explorar estrategias de prueba para MLS, cubriendo diversos niveles de prueba y la aplicación de pruebas basadas en riesgos.
- Capítulo 5: 180 minutos – Pruebas de Datos de Entrada para Sistemas de Machine Learning
- Aprender a probar y validar los datos de entrada para MLS, incluyendo técnicas para detectar sesgos, verificar la corrección del etiquetado, evaluar la representatividad de los datos y probar el pipeline de datos.
- Capítulo 6: 225 minutos – Pruebas de Modelos para Sistemas de Machine Learning
- Descubrir enfoques de prueba para mitigar riesgos en modelos de ML, incluyendo revisiones de documentación, pruebas de rendimiento funcional de ML y detección de sobreajuste, subajuste y deriva.
- Aprender enfoques de prueba avanzados, incluyendo pruebas adversariales, pruebas A/B, pruebas comparativas back-to-back y el uso de ataques para descubrir debilidades del modelo.
- Obtener comprensión práctica de las pruebas metamórficas, incluyendo cómo derivar y aplicar casos de prueba donde los oráculos de prueba tradicionales son insuficientes.
- Capítulo 7: 30 minutos – Pruebas de Desarrollo de Machine Learning

- Aprender enfoques y estrategias de prueba para mitigar riesgos durante el desarrollo y despliegue de MLS, para respaldar un comportamiento robusto del sistema en entornos de producción.

1 Introducción a la Inteligencia Artificial - 120 minutos

Palabras clave: Ninguna

Palabras clave específicas de AI: sistema basado en AI, inteligencia artificial, AI general, machine learning, ML development framework, AI estrecha, super AI

Objetivos de Aprendizaje para el Capítulo 1:

1.1 Introducción a la AI - AI-1.1.1 (K2) Diferenciar entre sistemas basados en AI y sistemas convencionales - AI-1.1.2 (K2) Distinguir entre AI estrecha, AI general y super AI - AI-1.1.3 (K2) Explicar los diferentes tipos de tecnologías de AI - AI-1.1.4 (K2) Explicar la AI generativa - AI-1.1.5 (K2) Comparar las opciones disponibles de hardware para implementar sistemas de machine learning - AI-1.1.6 (K2) Comparar las opciones para el desarrollo y alojamiento de modelos de AI - AI-1.1.7 (K2) Resumir la funcionalidad proporcionada por los ML development frameworks - AI-1.1.8 (K2) Explicar cómo las regulaciones y los estándares afectan al desarrollo y las pruebas de sistemas basados en AI

1.1 Introducción a la AI

Este capítulo presenta las distinciones clave entre los sistemas convencionales y los basados en AI, centrándose en sus enfoques de diseño, adaptabilidad y explicabilidad. Describe el espectro de capacidades de la AI, que abarca desde la AI estrecha hasta la AI general y la super AI, y describe las tecnologías fundamentales como el ML, el aprendizaje profundo y la GenAI. El capítulo describe el hardware común y los entornos de desarrollo para sistemas basados en AI, así como los frameworks de ML más

populares. Finalmente, considera las regulaciones esenciales y los estándares técnicos que guían el desarrollo y despliegue responsable de la AI en diversos dominios.

1.1.1 Sistemas Basados en AI y Sistemas Convencionales

Los sistemas informáticos convencionales se programan típicamente utilizando lenguajes imperativos, donde los desarrolladores humanos definen explícitamente instrucciones paso a paso, incluyendo construcciones como sentencias if-then-else y bucles. Este enfoque determinista ayuda a que el comportamiento del sistema sea predecible y transparente, facilitando a los humanos la comprensión de cómo las entradas producen diferentes salidas. En contraste, la mayoría de los sistemas basados en AI, particularmente aquellos que utilizan ML, no siguen reglas predefinidas. En su lugar, analizan patrones en los datos para determinar cómo responder a nuevas entradas. Por ejemplo, un sistema de reconocimiento de imágenes basado en AI entrenado para identificar gatos no se basa en reglas codificadas explícitamente. En su lugar, aprende de un conjunto de datos de imágenes de gatos, extrayendo patrones que luego aplica a imágenes no vistas para clasificarlas con precisión.

Una diferencia fundamental entre los sistemas convencionales y los basados en AI es cómo abordan la resolución de problemas. Muchos sistemas basados en AI se basan en razonamiento probabilístico, inferencia estadística y reconocimiento de patrones para generar resultados. Esto permite que los modelos de AI manejen formas complejas de incertidumbre y ambigüedad de manera más efectiva, lo que resulta en salidas que no siempre son predecibles.

Un desafío clave en los sistemas basados en AI es la explicabilidad. Muchos modelos de AI, particularmente las arquitecturas de aprendizaje profundo, pueden contener miles de millones de parámetros, lo que hace que su funcionamiento interno sea difícil de interpretar para los humanos. Esta naturaleza de "caja negra" genera preocupaciones en dominios críticos como la salud, las finanzas, la defensa y el transporte, donde comprender por qué un modelo de AI tomó una decisión particular es crucial.

Lograr transparencia (véase 2.1.1) y explicabilidad en la toma de decisiones impulsada por AI se ha convertido en un enfoque crítico en la regulación de la AI.

Otra distinción significativa es la adaptabilidad. Los sistemas convencionales son estáticos y típicamente requieren actualizaciones manuales para incorporar nuevo conocimiento o responder a cambios en el entorno. Los sistemas basados en AI, en contraste, pueden ser autoajustables, mejorando continuamente su rendimiento a medida que encuentran nuevos datos. Esta adaptabilidad hace que la AI sea particularmente poderosa en entornos dinámicos. La adaptabilidad también requiere monitorización continua para mantener la alineación permanente con los requisitos fundamentales.

1.1.2 AI Estrecha, AI General y Super AI

La AI estrecha, también conocida como AI débil, está diseñada para realizar tareas específicas y representa todos los sistemas de AI desplegados en uso actualmente. Los sistemas basados en AI estrecha operan dentro de un dominio limitado y pueden ser altamente eficientes en la resolución de problemas especializados, como el reconocimiento de imágenes, el procesamiento del habla y la traducción de idiomas. Sin embargo, carecen de la capacidad de generalizar más allá de las funciones que han aprendido. Por ejemplo, un sistema basado en AI que destaca en el reconocimiento de rostros no puede realizar traducción de idiomas a menos que sea reentrenado explícitamente para hacerlo. La AI de frontera, un subconjunto de la AI estrecha, representa la forma más avanzada de estos sistemas, empujando los límites de las capacidades actuales con modelos de GenAI. La AI de frontera incluye sistemas a gran escala con capacidades de toma de decisiones altamente autónomas; sin embargo, siguen siendo específicos para tareas y aún no han alcanzado la versatilidad de la AI general.

La AI general, también conocida como AI fuerte, se refiere a un sistema basado en AI que posee la capacidad de realizar la mayoría de las tareas intelectuales que un humano puede realizar. La AI general sería capaz de comprender, aprender y

aplicar conocimiento en una amplia gama de tareas sin necesidad de ser reentrenada para cada nueva tarea. Este tipo de AI exhibiría razonamiento y adaptabilidad similares a los humanos, capaz de resolver problemas desconocidos en diversos dominios, de manera muy similar a como lo hacen los humanos. A pesar del progreso significativo en AI, ningún sistema basado en AI posee actualmente inteligencia general.

La super AI (superinteligencia artificial) es la forma de AI en la que un sistema basado en AI mejora continuamente por sí mismo sin necesidad de intervención o control humano. Para que la super AI sea posible, el acceso a Internet no es estrictamente necesario, pero dicho acceso podría expandir significativamente sus capacidades e influencia. Superaría a la inteligencia humana y a la AI general, lo que muchos creen que representaría un riesgo existencial para la humanidad. El punto en el que los sistemas basados en AI transicionarían de la AI general a la super AI, si llegara a ocurrir, se conoce comúnmente como la singularidad tecnológica.

1.1.3 Diferentes Tipos de Tecnologías de AI

La Inteligencia Artificial abarca una gama de tecnologías, cada una adaptada a tareas y desafíos específicos. Una de las ramas fundamentales de la AI es el ML, que permite a los sistemas aprender de los datos y construir modelos sin programación explícita. Mientras que algunos MLs pueden adaptarse y mejorar continuamente con nuevos datos a lo largo de su vida útil, otros operan con el conocimiento que aprendieron inicialmente y requieren reentrenamiento explícito para actualizar sus capacidades. El ML incluye varios enfoques: - El aprendizaje supervisado utiliza datos etiquetados y algoritmos, como la regresión lineal y los árboles de decisión, para tareas como la predicción y la clasificación. - El aprendizaje no supervisado descubre patrones en datos no etiquetados mediante técnicas como el clustering (utilizando algoritmos de clustering). - El aprendizaje por refuerzo permite a los agentes inteligentes aprender comportamientos óptimos a través de interacciones de prueba y error con su entorno.

Las tecnologías clave de ML incluyen redes neuronales, modelos bayesianos, máquinas de vectores de soporte (SVM) y bosques aleatorios. Véase el Capítulo 3 para más información sobre ML.

El Aprendizaje Profundo (DL), un subconjunto de los métodos de ML, utiliza redes neuronales profundas para resolver problemas complejos. Por ejemplo: - Las redes neuronales convolucionales (CNN) son altamente efectivas para el reconocimiento de imágenes y la detección de objetos. - Las redes neuronales recurrentes (RNN) se especializan en el procesamiento de datos secuenciales, como texto o series temporales. - Los transformers manejan dependencias de largo alcance en secuencias, potenciando modelos para el procesamiento del lenguaje natural y vision transformers para imágenes.

La GenAI se basa en estas tecnologías para crear nuevo contenido, incluyendo texto, imágenes y audio (véase 1.1.4). Esto es impulsado por modelos como los LLM, que combinan redes neuronales profundas (DNN) con procesamiento del lenguaje natural (NLP) para analizar y generar lenguaje similar al humano.

Otras tecnologías de AI especializadas incluyen: - NLP para el análisis del lenguaje, incluyendo tareas como el análisis de sentimiento y la traducción automática. - Visión por computador para el análisis de datos visuales, apoyando aplicaciones como el reconocimiento facial y la robótica. - Lógica difusa para el razonamiento bajo incertidumbre. - Algoritmos de búsqueda para resolver problemas de optimización, como la navegación, y la toma de decisiones estratégicas. - Sistemas de razonamiento basados en reglas, o sistemas expertos, para el soporte estructurado de decisiones.

Aunque la integración completa de estas tecnologías de AI sigue siendo limitada, nuevos desarrollos como los LLM demuestran el potencial de combinar diferentes tecnologías de AI en sistemas inteligentes unificados. La AI agéntica extiende estas tecnologías a través de agentes autónomos que planifican, razonan y actúan de forma independiente para alcanzar objetivos en entornos dinámicos.

1.1.4 AI Generativa

La AI Generativa (GenAI) se refiere a sistemas basados en AI especializados en la creación de nuevo contenido, como texto, imágenes, vídeo, música o datos complejos, mientras que muchos también admiten clasificación y predicción. Estos sistemas aprenden de grandes cantidades de datos para producir salidas que se asemejan a sus datos de entrenamiento, habilitando una amplia gama de aplicaciones creativas y prácticas.

Las principales tecnologías detrás de la GenAI incluyen las redes generativas adversariales (GANs), los modelos de difusión y los transformers. Las GANs utilizan dos redes neuronales en competencia para crear datos sintéticos altamente realistas. Los modelos de difusión generan contenido añadiendo gradualmente ruido a los datos y luego eliminándolo, produciendo salidas de alta calidad. Los modelos transformer, que sustentan los LLM, utilizan mecanismos de autoatención para generar texto coherente y contextualmente relevante, y se están adaptando cada vez más para tareas multimodales.

Más allá de sus fundamentos técnicos, los sistemas de GenAI plantean preocupaciones sociales y éticas significativas. El uso indebido es una preocupación importante: estos modelos pueden ser explotados para crear deepfakes, difundir desinformación o generar contenido fraudulento convincente, socavando así la confianza en los medios digitales y el discurso público. La facilidad de producir contenido sintético amplifica los riesgos relacionados con la privacidad, la seguridad y la manipulación, pero puede aportar beneficios notables y abre oportunidades de innovación en entretenimiento, marketing, educación e investigación.

El impacto en el empleo también es notable, particularmente entre las profesiones de cuello blanco. A medida que la GenAI automatiza tareas como la escritura, el diseño, la programación e incluso la documentación legal o médica, existe un debate creciente sobre el posible desplazamiento laboral. Si bien estas tecnologías pueden aumentar la productividad y fomentar nuevas oportunidades creativas,

también pueden provocar interrupciones en la fuerza laboral y requerir una recualificación generalizada.

La sostenibilidad es otra cuestión apremiante. El entrenamiento y la ejecución de grandes modelos de GenAI implican recursos computacionales sustanciales, lo que resulta en un alto consumo de energía y una huella de carbono significativa. Este impacto ambiental ha provocado llamados a diseños de modelos más eficientes e infraestructuras más ecológicas.

La mayoría de las herramientas prácticas de GenAI actuales se basan en modelos fundacionales, que luego se ajustan finamente para aplicaciones específicas. El campo también avanza hacia modelos multimodales capaces de procesar y generar contenido en texto, imágenes y audio, habilitando sistemas basados en AI más ricos y flexibles. Marcos regulatorios, como el EU AI Act [EU AI Act], están emergiendo para guiar el desarrollo y uso responsable de estas tecnologías.

1.1.5 Hardware para Sistemas de Machine Learning

Se utiliza una variedad de hardware para los MLS. Diferentes tipos de hardware pueden usarse para el entrenamiento y la inferencia. Por ejemplo, un modelo de reconocimiento de voz puede ejecutarse en un smartphone de gama baja, aunque puede necesitarse acceso a la potencia de la computación en la nube para entrenarlo.

El ML típicamente se beneficia de hardware que admita lo siguiente:

- La capacidad de trabajar con grandes estructuras de datos.
- Procesamiento masivamente paralelo (concurrente), por ejemplo, para soportar la multiplicación de matrices.
- Aritmética de baja precisión (cuantización): Este enfoque utiliza menos bits para el cálculo (por ejemplo, 4 bits en lugar de 32 bits), lo que resulta en un procesamiento más rápido, menor consumo de energía, chips más pequeños y económicos, y requisitos de ancho de banda reducidos.

Las unidades centrales de procesamiento (CPU) de propósito general proporcionan soporte para operaciones complejas con alta precisión que no son típicamente requeridas para aplicaciones de ML, pero normalmente proporcionan solo

unos pocos núcleos. Como resultado, su arquitectura es menos eficiente para entrenar y ejecutar modelos de ML en comparación con las unidades de procesamiento gráfico (GPU), que tienen miles de núcleos y están diseñadas para realizar procesamiento gráfico masivamente paralelo, aunque relativamente simple. En consecuencia, las GPU típicamente superan a las CPU en aplicaciones de ML, aunque las CPU generalmente funcionan a velocidades de reloj más altas. Para trabajos de ML a pequeña escala, las GPU generalmente ofrecen la mejor opción.

Algún hardware está diseñado específicamente para AI, como los Circuitos Integrados de Aplicación Específica (ASIC) contruidos a propósito y los dispositivos System-on-a-Chip. Estas soluciones específicas para AI cuentan con múltiples núcleos, gestión de datos especializada y la capacidad de realizar procesamiento en memoria. Son más adecuados para la computación en el borde (edge computing), mientras que el entrenamiento del modelo de ML se realiza en la nube utilizando hardware especializado.

Las arquitecturas de hardware específicas para AI continúan desarrollándose. Esto incluye los procesadores neuromórficos, que no utilizan la arquitectura tradicional de von Neumann, sino diseños inspirados en el cerebro que imitan las estructuras neuronales.

1.1.6 Desarrollo y Alojamiento de Modelos de AI

Los sistemas basados en AI pueden adquirirse de proveedores externos o desarrollarse de forma privada dentro de una organización. Los sistemas basados en AI típicamente dependen de modelos preentrenados y pueden desplegarse en las instalaciones (on-premises) o en la nube, con los modelos alojados ya sea en las instalaciones o en la nube, donde las opciones basadas en la nube a menudo se acceden como servicio (AlaaS). Los sistemas basados en AI de terceros típicamente vienen como modelos preentrenados o AI como Servicio (AlaaS), permitiendo un despliegue más rápido y un menor tiempo de comercialización. Alternativamente, los sistemas basados en AI privados (on-premises o configuraciones personalizadas en la nube) pueden adaptarse mejor a requisitos específicos, pero su

desarrollo probablemente requerirá habilidades especializadas, ya sea a través de expertos internos o equipos subcontratados.

El desarrollo local (codificación y entrenamiento local) permite un control directo y privacidad. Los modelos pequeños, como árboles de decisión o redes neuronales compactas, pueden desarrollarse en computadoras personales, mientras que los modelos de tamaño medio pueden requerir GPU dedicadas. Para modelos a gran escala, se necesitan clústeres de servidores de alto rendimiento on-premises con sus costos asociados de energía, refrigeración y hardware.

El desarrollo en la nube ofrece una flexibilidad significativa. Las nubes públicas, en particular, proporcionan entornos preconfigurados con precios de pago por uso, lo que limita la inversión inicial en hardware y escala fácilmente. En contraste, las nubes privadas pueden proporcionar mayor seguridad y privacidad para las aplicaciones que lo requieran, pero este control exige una mayor inversión inicial en infraestructura.

Muchas organizaciones adoptan enfoques híbridos, incluyendo el desarrollo de prototipos localmente antes de escalar a infraestructura en la nube, manteniendo componentes sensibles on-premises, como la preparación de datos privados, y aprovechando recursos en la nube para tareas de computación intensiva.

Los modelos de AI pueden alojarse en diversos entornos, que van desde configuraciones locales hasta plataformas basadas en la nube. El alojamiento local implica ejecutar modelos más pequeños en computadoras personales o smartphones, ofreciendo privacidad y eliminando costos de licencias en la nube, aunque esto proporciona capacidades de hardware limitadas. Para modelos de AI más grandes, las organizaciones pueden establecer servidores dedicados, que requieren una inversión inicial significativa pero proporcionan un mayor control.

El alojamiento en la nube de modelos de AI puede ser en nubes públicas o privadas. Los servicios de nube pública proporcionan acceso escalable a infraestructura robusta y

potente, eliminando preocupaciones de mantenimiento y siendo ideales para cargas de trabajo fluctuantes. Las nubes privadas ofrecen beneficios similares, con mayor seguridad y opciones de personalización, y son gestionadas internamente o a través de proveedores dedicados, aunque a un mayor costo.

Los enfoques híbridos combinan estos métodos, permitiendo a las organizaciones ejecutar algunas operaciones localmente mientras aprovechan la elasticidad de la nube para tareas intensivas.

Las soluciones óptimas de desarrollo y alojamiento, que típicamente se deciden por separado, dependen de factores como el tamaño del modelo, la complejidad, los requisitos de rendimiento, las restricciones presupuestarias, las consideraciones de seguridad y privacidad de datos, las necesidades de despliegue y los requisitos regulatorios. Algunas organizaciones adoptan estrategias multinivel para equilibrar eficiencia y control.

1.1.7 Frameworks de Desarrollo de Machine Learning

Los ML development frameworks proporcionan un conjunto de herramientas para construir y entrenar modelos de ML. La funcionalidad típica proporcionada por estos frameworks incluye:

- Manejo de datos: Ayudan con la carga, preprocesamiento y gestión de los datos utilizados para entrenar y probar el modelo. Esto puede incluir la limpieza, el formateo y la transformación de los datos a un formato adecuado para el modelo elegido.
- Construcción de modelos: Estos frameworks ofrecen bibliotecas de algoritmos de ML y herramientas para diseñar la arquitectura del modelo de ML construido. Esto incluye especificar el tipo de modelo (por ejemplo, red neuronal, árbol de decisión), el número de capas y conexiones, y las operaciones matemáticas realizadas dentro del modelo.
- Entrenamiento y optimización: Los frameworks proporcionan algoritmos que ajustan iterativamente los parámetros internos del modelo basándose en los datos de entrenamiento y en el resultado deseado. El objetivo es optimizar el rendimiento del modelo en la realización de la tarea deseada (por ejemplo,

clasificación, regresión ML). Algunos frameworks pueden soportar entrenamiento distribuido y mejorar o ajustar finamente modelos preentrenados. - Evaluación: Ofrecen herramientas para evaluar qué tan bien se desempeña el modelo entrenado con datos no vistos. Esto puede incluir la medición de la exactitud, la precisión y la exhaustividad para tareas de clasificación, o tasas de error para tareas de regresión ML (véase 3.1.1). - Despliegue: Algunos frameworks proporcionan capacidades para desplegar el modelo entrenado para su uso en el mundo real. Esto podría incluir la conversión del modelo a un formato adecuado para la integración con aplicaciones web, dispositivos móviles, dispositivos de borde o sistemas embebidos.

Estos frameworks pueden operar a diferentes niveles de abstracción. Algunos ofrecen una interfaz de programación de aplicaciones (API) de bajo nivel, proporcionando a los desarrolladores más control sobre la construcción del modelo pero requiriendo más experiencia en codificación. Otros ofrecen una API de alto nivel, simplificando la creación del modelo pero ofreciendo menos opciones de personalización.

Diferentes frameworks pueden centrarse en diferentes dominios de aplicación. Algunos son de propósito general y admiten una amplia gama de áreas de aplicación. En contraste, otros son más especializados, centrándose en áreas específicas como el reconocimiento de imágenes, el reconocimiento de voz y la traducción de idiomas.

La selección del framework más apropiado puede depender de varios factores, como: - el área de aplicación; - la necesidad de una interfaz fácil de usar para la creación rápida de prototipos; - la configurabilidad para modelos complejos; - la experiencia de los usuarios; - consideraciones de despliegue, ya que algunos frameworks son más adecuados para entornos con recursos limitados; - nivel de soporte (de la comunidad); - madurez del ecosistema.

1.1.8 Regulaciones y Estándares para AI

Las regulaciones y estándares de AI son cruciales para el desarrollo, despliegue y uso responsable de la AI. El objetivo principal es fomentar la confianza en la AI y ayudar a

promover la realización de los beneficios de la AI mientras se mitigan los posibles daños. Idealmente, el cumplimiento de dichas regulaciones y estándares debería garantizar que los sistemas basados en AI sean seguros, justos, transparentes, sostenibles, responsables, éticos y utilizados de manera responsable.

A nivel internacional, los OECD AI Principles [OECD AI] y el informe de la ONU sobre Governing AI for Humanity [UN Gov AI] sirven como instrumentos influyentes de derecho indicativo (soft law) que fomentan una comprensión compartida de la gestión responsable de la AI. Actúan como una brújula para los gobiernos nacionales y las organizaciones mientras formulan sus propias estrategias de AI. Estos principios enfatizan una AI centrada en el ser humano, consideraciones éticas y la importancia de la cooperación internacional.

El EU AI Act representa un hito regulatorio, demostrando un enfoque basado en riesgos para la regulación de la AI. Al categorizar los sistemas basados en AI por riesgo, desde mínimo hasta inaceptable, las regulaciones se adaptan en consecuencia. Los sistemas de alto riesgo, particularmente aquellos que impactan en los derechos fundamentales o la seguridad, enfrentan requisitos estrictos que abarcan pruebas rigurosas, gobernanza de datos y supervisión humana. Las sanciones financieras sustanciales por incumplimiento, basadas en un porcentaje de la facturación global, subrayan el compromiso de la UE con la aplicación de la ley. En contraste, muchas naciones fuera de la UE están adoptando un enfoque más permisivo, favoreciendo regulaciones menos restrictivas para fomentar la innovación.

Los estándares técnicos, desarrollados por organizaciones como ISO e IEEE, son cruciales para traducir las aspiraciones de alto nivel en implementaciones prácticas. Proporcionan especificaciones técnicas concretas y mejores prácticas, cerrando la brecha entre la política y la práctica. Por ejemplo, ISO/IEC TR 29119-11 proporciona orientación detallada sobre las pruebas de sistemas basados en AI, un elemento crítico para demostrar el cumplimiento regulatorio. Mientras tanto, la serie ISO/IEC 42119 se está desarrollando para cubrir diversos aspectos de las pruebas de sistemas basados en AI.

Además, están surgiendo regulaciones sectoriales en áreas como la salud y las finanzas, reconociendo los riesgos únicos que plantea la AI en estos dominios.

Para navegar eficazmente este panorama en evolución de la gobernanza de la AI, el diálogo y la colaboración continuos son fundamentales. Los gobiernos, la industria, el ámbito académico y la sociedad civil deben participar activamente para lograr un enfoque armonizado y efectivo de la gobernanza de la AI a nivel mundial. Además, dada la naturaleza dinámica de la AI, las regulaciones y los estándares deben revisarse y actualizarse periódicamente para seguir siendo relevantes y efectivos en la orientación del desarrollo, despliegue y uso responsable de la AI.

2 Características de Calidad para Sistemas Basados en IA - 45 minutos

Palabras clave: Adaptabilidad funcional, corrección funcional de IA, intervenibilidad, robustez de IA, seguridad (safety), mitigación de riesgos sociales y éticos, transparencia, controlabilidad por el usuario

Palabras clave específicas de IA: Ninguna

Objetivos de Aprendizaje del Capítulo 2: 2.1 Características de Calidad para Sistemas Basados en IA - AI-2.1.1 (K2) Clasificar comportamientos de sistemas basados en IA según las características de calidad definidas en ISO/IEC 25059 - AI-2.1.2 (K2) Explicar las consideraciones especiales que surgen cuando se utiliza IA en sistemas relacionados con la seguridad (safety)

2.2 Criterios de Aceptación para Sistemas Basados en IA - AI-2.2.1 (K2) Dar ejemplos de criterios de aceptación para sistemas basados en IA

2.1 Características de Calidad para Sistemas Basados en IA

Esta sección cubre las características de calidad específicas de IA descritas en ISO/IEC 25059, que extiende los modelos

tradicionales de calidad de software para abordar aspectos únicos de los sistemas basados en IA. Introduce características nuevas y adaptadas, incluyendo la corrección funcional de IA, la adaptabilidad funcional, la controlabilidad por el usuario, la transparencia, la robustez de IA y la intervenibilidad, junto con la mitigación de riesgos sociales y éticos. También se abordan los principales desafíos de seguridad de la IA, como las especificaciones vagas, el no determinismo, el autoaprendizaje, la explicabilidad limitada y las normativas en evolución, con énfasis en su impacto en las pruebas y la regulación.

2.1.1 Características de Calidad Específicas de IA

ISO/IEC 25059 extiende el modelo de calidad de ISO/IEC 25010 para abordar consideraciones específicas de IA. Esta extensión evalúa los sistemas basados en IA desde dos perspectivas: calidad del producto y calidad en uso. Desde una perspectiva de pruebas, estas características de calidad influyen directamente en cómo se definen los objetivos de prueba, cómo se formulan los criterios de aceptación y cómo se interpretan los resultados de las pruebas para sistemas basados en IA. Las características nuevas y modificadas, en comparación con ISO/IEC 25010, incluyen:

- Corrección funcional de IA (calidad del producto): Los sistemas basados en IA, especialmente aquellos que utilizan ML probabilístico, no pueden garantizar una precisión perfecta. Dado que se espera cierta tasa de error en las salidas de IA, el concepto de corrección funcional se ha ajustado en consecuencia. ISO/IEC 25059 evalúa la corrección funcional considerando tanto las salidas correctas como las incorrectas y definiendo umbrales aceptables para resultados incorrectos, reflejando la variabilidad inherente en las salidas de sistemas basados en IA (véase 3.3).
- Adaptabilidad funcional (calidad del producto): una nueva subcaracterística de la idoneidad funcional. La capacidad del sistema basado en IA para adaptarse de forma autónoma a cambios en su entorno operativo después de ser desplegado.

- Controlabilidad por el usuario (calidad del producto): una nueva subcaracterística de la capacidad de interacción (nótese que la capacidad de interacción es en sí misma un nuevo término que reemplaza a usabilidad en la versión 2023 de ISO/IEC 25010). Una propiedad de un sistema basado en IA tal que un humano u otro agente externo puede intervenir en su funcionamiento de manera oportuna.
- Transparencia (calidad del producto y calidad en uso): una nueva subcaracterística de la capacidad de interacción y una nueva subcaracterística de satisfacción. Se refiere al grado en que se comunica información apropiada sobre el sistema basado en IA a las partes interesadas (véase 6.1.2).
- Robustez de IA (calidad del producto): una nueva subcaracterística de la fiabilidad. Describe la capacidad de un sistema basado en IA para mantener su nivel de corrección funcional de IA independientemente de las circunstancias, como la presencia de datos de entrada sesgados, adversariales o inválidos, interferencias externas, condiciones ambientales adversas y uso indebido por parte del operador.
- Intervenibilidad (calidad del producto): una nueva subcaracterística de la seguridad (security). El grado en que un operador puede intervenir en el funcionamiento de un sistema basado en IA de manera oportuna para prevenir daños o peligros.
- Mitigación de riesgos sociales y éticos (calidad en uso): una nueva subcaracterística de 'Libertad frente al riesgo'. Considera muchas áreas para mitigar riesgos sociales y éticos, incluyendo la rendición de cuentas, la equidad y no discriminación, la responsabilidad profesional, la promoción de valores humanos, la privacidad, la seguridad (safety y security), el control humano de la tecnología, la participación y el desarrollo comunitario, el diseño centrado en el ser humano, el respeto al estado de derecho, el respeto a las normas internacionales de comportamiento, la sostenibilidad ambiental y las prácticas laborales.

2.1.2 IA y Seguridad (Safety)

Los sistemas relacionados con la seguridad tienen el potencial de causar lesiones o daños a personas, propiedades o al medio ambiente. Desarrollar y probar sistemas relacionados con la seguridad que no utilizan IA puede requerir mucho esfuerzo, pero es viable; sin embargo, para los sistemas basados en IA, existen varios desafíos adicionales:

- **Especificaciones:** En los sistemas tradicionales relacionados con la seguridad, los requisitos se definen para el sistema completo y se refinan hasta que el desarrollador pueda transformarlos en código. Los requisitos para muchos sistemas basados en IA a menudo comienzan con objetivos vagos y luego se proporcionan implícitamente a través de los datos de entrenamiento que codifican patrones, reglas y objetivos, sin formalizar completamente cada detalle de antemano. Esto puede significar que la trazabilidad necesaria desde los requisitos hasta la implementación es inadecuada para los sistemas basados en IA.
- **No determinismo:** Esta característica de muchos sistemas basados en IA hace que sea inherentemente difícil garantizar el comportamiento preciso de estos sistemas. Incluso los modelos rigurosamente probados pueden exhibir comportamientos inesperados debido a factores como la generación de números aleatorios o ligeras variaciones en los valores de entrada.
- **Autoaprendizaje:** Las pruebas rigurosas se utilizan para demostrar la integridad de seguridad de un sistema antes de su despliegue. Para los sistemas basados en IA con autoaprendizaje, esto se ve socavado ya que el comportamiento del sistema se aleja progresivamente del comportamiento originalmente probado. Gestionar cómo aprende el modelo y los datos que utiliza puede ayudar a veces a evitar la aparición de nuevos comportamientos problemáticos. Alternativamente, se pueden implementar salvaguardas de seguridad para ayudar

a prevenir que el modelo aprenda o tome decisiones que puedan comprometer la seguridad (por ejemplo, un componente de moderación de contenido para filtrar prompts).

- **Explicabilidad y Transparencia:** Para los sistemas relacionados con la seguridad, es esencial comprender cómo y por qué el sistema toma decisiones. Sin embargo, los procesos de toma de decisiones de los sistemas basados en IA a menudo no son transparentes. Las técnicas de IA explicable, como LIME (Local interpretable model-agnostic explanations), pueden proporcionar información sobre el razonamiento del sistema basado en IA; sin embargo, no están ampliamente disponibles y pueden comprometer el rendimiento del sistema.
- **Regulaciones en evolución:** El panorama regulatorio para los sistemas basados en IA relacionados con la seguridad está en constante evolución. El uso de IA actualmente no está incluido en las normas internacionales maduras de seguridad funcional, y algunas de estas normas incluso prohíben su uso en dichos sistemas. El EU AI Act [EU AI Act] (véase 1.1.8) clasifica los sistemas de IA utilizados como componentes de seguridad (como en aviación, dispositivos médicos o automoción) como de alto riesgo e impone requisitos estrictos sobre su desarrollo y pruebas.

2.2 Criterios de Aceptación para Sistemas Basados en IA

Esta sección describe los criterios de aceptación relacionados con las características de calidad especificadas en la norma ISO/IEC 25059, así como con la seguridad (safety). Para los sistemas basados en IA, los criterios de aceptación a menudo necesitan ser estadísticos, probabilísticos o basados en umbrales en lugar de binarios, lo que introduce desafíos adicionales en las pruebas.

2.2.1 Criterios de Aceptación para Sistemas Basados en IA

Al evaluar la calidad de un sistema basado en IA, es esencial considerar tanto las características de calidad funcionales como las no funcionales. Esto ayuda a confirmar que el sistema basado en IA funciona según lo previsto y satisface requisitos de calidad más amplios. Las normas ISO/IEC 25010 e ISO/IEC 25059 proporcionan un marco integral para definir la calidad del software. En esta sección, el enfoque se centra en los criterios de aceptación asociados con las características de calidad específicas de IA (es decir, las definidas en ISO/IEC 25059) y la seguridad (véase 2.1.2).

La siguiente tabla enumera ejemplos de criterios de aceptación para la seguridad y cada una de las características de calidad definidas en la norma ISO/IEC 25059.

Característica	Ejemplos de Criterios de Aceptación
Corrección funcional de IA (véase 3.3)	Precisión del 95% para un sistema de reconocimiento de imágenes. Recall del 90% para un sistema de predicción de defectos.
Adaptabilidad funcional	Un máximo de 20 segundos para que el sistema de gestión del motor se adapte cuando cruza un umbral de altitud especificado. Un servicio de streaming de vídeo debe ajustar su página de inicio para recomendar al menos un 40% de documentales después de que un usuario vea tres documentales completos en una sola sesión.
Controlabilidad por el usuario	Un supervisor puede tomar el control de un dron autónomo en 0,5 segundos cuando este envía una señal de socorro debido a la pérdida de su ubicación GPS. El sistema de control agrícola notifica al agricultor cuando el rendimiento visual del sensor se degrada en más del 30%, permitiendo una anulación manual inmediata; se desactiva

Característica	Ejemplos de Criterios de Aceptación
	completamente si la degradación supera el 50% sin respuesta del usuario.
Transparencia	Se proporciona información suficiente sobre el modelo de ML de terceros y la procedencia de sus datos de entrenamiento para cumplir con los requisitos de la norma empresarial correspondiente. El panel de operaciones del sistema y la API deben proporcionar un endpoint que devuelva el identificador de versión único del modelo de predicción actualmente desplegado y un enlace a su documentación correspondiente.
Robustez de IA	El tiempo de respuesta para las predicciones del sistema de alertas de penetración de seguridad basado en IA se mantiene por debajo de 1 segundo cuando el acceso a la base de datos central de vulnerabilidades se interrumpe durante 30 segundos. El dispositivo de IA en el borde (edge) debe hacer una transición automática a un modo de inferencia de menor fidelidad y menor consumo de energía (en lugar de fallar) cuando su temperatura interna de operación excede los 85°C durante un período continuo de 10 segundos.
Intervenibilidad	Si un robot traspasa su zona de seguridad, la línea de producción puede ser detenida en 0,5 segundos después de que se inicie el apagado. Para prevenir posibles apagones, el sistema de gestión de la red eléctrica debe proporcionar una ventana de confirmación de 30 segundos, durante la cual un ingeniero puede vetar cualquier acción propuesta por la IA clasificada como 'crítica' antes de que se ejecute automáticamente.
Mitigación de riesgos sociales y éticos	El sistema automatizado de sentencias carcelarias no discrimina entre grupos raciales según la métrica de equidad especificada. El chatbot debe superar una

Característica	Ejemplos de Criterios de Aceptación
	evaluación interna de "red teaming" con una puntuación del 95% o superior, demostrando su rechazo a generar contenido que promueva la violencia, la autolesión o el discurso de odio.
Seguridad (Safety)	Los componentes no basados en IA del sistema de control de dirección basado en IA cumplen con ISO 26262-6 en el nivel ASIL C. El 100% de las relaciones entre entradas y salidas del modelo de ML en el sistema de control de la central nuclear pueden ser mapeadas con una precisión promedio no inferior al 99,9% mediante una herramienta de explicabilidad. Las señales de control que exceden los límites de seguridad especificados en más del 10% son analizadas y reguladas en 0,15 segundos después de ser detectadas por el subsistema de monitorización de seguridad.

3 Machine Learning - 375 minutos

Palabras clave: Cobertura de neuronas por k-multisección, criterios de rendimiento funcional de ML, métrica de rendimiento funcional de ML, modelo de ML, cobertura de límites de neuronas, cobertura de neuronas, perceptrón

Palabras clave específicas de IA: Asociación, clasificación, clustering, preparación de datos, machine learning, algoritmo de ML, framework de desarrollo de ML, flujo de trabajo de ML, modelo preentrenado, regresión de ML, aprendizaje por refuerzo, aprendizaje supervisado, aprendizaje no supervisado

Objetivos de Aprendizaje del Capítulo 3: 3.1 Introducción al Machine Learning - AI-3.1.1 (K2) Distinguir entre las diferentes formas de ML - AI-3.1.2 (K2) Resumir el flujo de trabajo utilizado para crear un sistema de ML - HO-3.1.3 (H2) Crear un modelo de ML - AI-3.1.4 (K2) Resumir

el uso de modelos preentrenados, el ajuste fino (fine-tuning) y la generación aumentada por recuperación (RAG)

3.2 Datos para Machine Learning - AI-3.2.1 (K2) Explicar las actividades relacionadas con la preparación de datos - HO-3.2.2 (H2) Realizar la preparación de datos para apoyar la creación de un modelo de ML - AI-3.2.3 (K2) Contrastar el uso de conjuntos de datos de entrenamiento, validación y prueba en el desarrollo de un modelo de ML

3.3 Métricas de Rendimiento Funcional de ML para Clasificación - AI-3.3.1 (K3) Calcular métricas comunes de rendimiento funcional de ML a partir de un conjunto dado de datos de matriz de confusión - HO-3.3.2 (H2) Evaluar un modelo de ML utilizando métricas de rendimiento funcional de ML seleccionadas - HO-3.3.3 (H2) Mostrar el impacto de diferentes combinaciones de modelos de ML y conjuntos de datos en el entrenamiento y comportamiento de los modelos

3.4 Redes Neuronales - AI-3.4.1 (K2) Explicar la estructura y el funcionamiento de una red neuronal profunda - HO-3.4.2 (H1) Experimentar la implementación de un perceptrón - AI-3.4.3 (K2) Describir las diferentes medidas de cobertura para redes neuronales

3.1 Introducción al Machine Learning

Esta sección presenta las principales categorías de algoritmos de ML: aprendizaje supervisado, no supervisado y por refuerzo, distinguiendo sus respectivos tipos de problemas y aplicaciones típicas. Describe el flujo de trabajo estándar para el desarrollo de modelos de ML, desde la definición de objetivos y la preparación de datos hasta el entrenamiento, la evaluación y el despliegue de modelos, con atención a la iteración y la integración del sistema. También se cubren aspectos prácticos como la creación práctica de modelos, el uso de modelos preentrenados, el ajuste fino (fine-tuning) y la generación aumentada por recuperación (RAG), destacando enfoques para adaptar y mejorar eficientemente los modelos de IA para nuevas tareas mientras se gestionan las limitaciones heredadas.

Comprender este flujo de trabajo es esencial para los testers, ya que diferentes actividades de prueba se aplican en diferentes etapas, y las fallas a menudo se originan en pasos anteriores como la preparación de datos o la selección del modelo.

3.1.1 Diferentes Formas de Machine Learning

Los algoritmos de ML se clasifican en aprendizaje supervisado, aprendizaje no supervisado y aprendizaje por refuerzo.

En el aprendizaje supervisado, los algoritmos entrenan modelos utilizando datos etiquetados, donde cada conjunto de entradas tiene una etiqueta de salida correspondiente (por ejemplo, imágenes etiquetadas como "perro" o "gato"). El modelo aprende a mapear entradas a salidas identificando patrones en los datos de entrenamiento. El aprendizaje supervisado se divide típicamente en: - Clasificación: Consiste en asignar entradas a clases predefinidas, como clasificar correos electrónicos como spam o no spam, o el reconocimiento de imágenes. - Regresión de ML: Consiste en predecir valores numéricos continuos, como estimar la edad de una persona basándose en datos de estilo de vida o pronosticar precios de acciones.

Nótese que el término regresión de ML, cuando se usa en el contexto de ML, difiere de su uso en otros syllabi de ISTQB®, donde regresión describe el problema de que las modificaciones de software causen defectos relacionados con cambios.

En el aprendizaje no supervisado, el algoritmo entrena modelos utilizando datos no etiquetados, infiriendo patrones o estructuras sin etiquetas de salida explícitas. El modelo agrupa entradas similares basándose en características compartidas. El aprendizaje no supervisado se categoriza típicamente en: - Clustering: Consiste en agrupar puntos de datos basándose en similitudes, como segmentar clientes en diferentes grupos para marketing dirigido. - Asociación: Consiste en identificar relaciones o dependencias entre atributos de datos, como encontrar patrones en el

comportamiento de compra de clientes para recomendar productos.

En el aprendizaje por refuerzo, el sistema basado en IA (un "agente inteligente") aprende interactuando con su entorno. El agente recibe retroalimentación positiva (recompensas) o retroalimentación negativa (penalizaciones) basada en el resultado de sus acciones, lo que le permite aprender de la experiencia en lugar de hacerlo a partir de un conjunto de datos. Los desafíos en el aprendizaje por refuerzo incluyen la configuración del entorno, el diseño de la función de recompensa y la selección de la mejor estrategia para alcanzar el objetivo deseado. Las aplicaciones incluyen robótica, vehículos autónomos y sistemas adaptativos como chatbots.

Cada enfoque de ML aborda diferentes tipos de problemas, y la elección depende de la naturaleza de los datos disponibles y de la tarea específica en cuestión.

3.1.2 Flujo de Trabajo de Machine Learning

Las actividades en el flujo de trabajo de ML, mostradas en la Figura 1, son:

Comprender los Objetivos: El propósito del modelo de ML es comprendido y acordado por las partes interesadas para verificar la alineación con las prioridades del negocio. Se definen los criterios de aceptación (incluyendo las métricas de rendimiento funcional de ML – véase 3.3) para el modelo desarrollado.

Seleccionar un Framework: Se selecciona un framework de desarrollo de ML adecuado (véase 1.1.7 para detalles sobre la funcionalidad proporcionada) basándose en los objetivos, los criterios de aceptación (véase 2.2) y las prioridades del negocio.

Seleccionar y Construir el Algoritmo: Se selecciona un algoritmo de ML basándose en diversos factores, incluyendo los objetivos, los criterios de aceptación y los datos disponibles (véase 3.2). El algoritmo puede ser codificado manualmente, pero a menudo se obtiene de una biblioteca

de software. El algoritmo se compila posteriormente, si es necesario.

Preparar y Probar los Datos: La preparación de datos (véase 3.2) comprende la adquisición de datos, el preprocesamiento de datos y la ingeniería de características. El análisis exploratorio de datos (EDA) puede realizarse junto con estas actividades. Los datos utilizados por el algoritmo y el modelo se basan en los objetivos y son utilizados por todas las actividades en el recuadro de 'generación y prueba del modelo' mostrado en la Figura 1. Los datos utilizados para entrenar, evaluar, ajustar y probar el modelo deben ser representativos de los datos que serán utilizados por el modelo en operación. Se realizan pruebas de los datos y de cualquier paso automatizado de preparación de datos (véase Capítulo 5).

Entrenar el Modelo: El algoritmo de ML seleccionado utiliza datos de entrenamiento para entrenar el modelo. Los parámetros que definen la estructura del modelo (por ejemplo, el número de capas de una red neuronal o la profundidad de un árbol de decisión) se pasan al algoritmo. Estos parámetros se conocen como hiperparámetros del modelo. Los parámetros que controlan el entrenamiento (por ejemplo, el número de iteraciones a utilizar cuando se entrena una red neuronal) también se pasan al algoritmo. Estos parámetros se conocen como hiperparámetros del algoritmo.

Evaluar el Modelo: El modelo se evalúa contra las métricas de rendimiento funcional de ML acordadas (véase 3.3), utilizando el conjunto de datos de validación, y los resultados se utilizan para mejorar el modelo en la actividad de 'ajustar el modelo'. En la práctica, típicamente se crean y entrenan varios modelos utilizando diferentes algoritmos (por ejemplo, random forests, SVM y redes neuronales) y varios conjuntos de datos de entrenamiento, y se elige la mejor combinación basándose en los resultados de la evaluación.

Ajustar el Modelo: Los resultados de la evaluación se utilizan para ajustar los hiperparámetros del modelo y los hiperparámetros del algoritmo. El modelo se vuelve a entrenar con estos ajustes para mejorar su rendimiento

funcional de ML. Las tres actividades de entrenamiento, evaluación y ajuste comprenden la 'generación del modelo', como se muestra en la Figura 1.

Probar el Modelo: Una vez que se ha generado un modelo aceptable mediante las actividades de 'generación del modelo', se prueba utilizando un conjunto de datos de prueba independiente para verificar que se cumplen los criterios de rendimiento funcional de ML acordados. Los resultados de las pruebas también se comparan con los de la evaluación. Si el rendimiento del modelo con datos de prueba independientes es significativamente inferior al obtenido durante la evaluación, puede ser necesario regresar a las actividades de 'generación del modelo', o incluso a la actividad de 'preparar y probar los datos' para entrenar un nuevo modelo. Además de las pruebas de rendimiento funcional de ML, también pueden realizarse pruebas no funcionales, como el tiempo para proporcionar una predicción. Típicamente, las pruebas en esta actividad son realizadas por ingenieros o científicos de datos; sin embargo, los testers con suficiente conocimiento del dominio y acceso a los recursos relevantes también pueden realizar estas pruebas.

Desplegar el Modelo: Una vez completada la 'generación y prueba del modelo', el modelo ajustado se reingeniería típicamente para su despliegue junto con su pipeline de datos. Esto generalmente se logra a través del framework de desarrollo de ML. Las plataformas objetivo pueden incluir sistemas embebidos y la nube, donde el modelo puede ser accedido a través de una API web. El modelo desplegado reingenierizado se prueba para verificar que aún cumple con sus criterios de aceptación.

Usar el Modelo: Una vez desplegado, el modelo se integra típicamente en un sistema operativo basado en IA más amplio. Los modelos pueden realizar predicciones por lotes programadas a intervalos de tiempo establecidos o ejecutarse en tiempo real bajo demanda.

Monitorizar y Ajustar el Modelo: Mientras el modelo está en uso, su situación puede evolucionar y el modelo puede desviarse de su rendimiento previsto (véase 6.1.7). Para

verificar que cualquier desviación sea identificada y gestionada, el modelo operativo se evalúa regularmente contra sus criterios de aceptación. Puede considerarse necesario crear un nuevo modelo mediante reentrenamiento con nuevos datos, reentrenamiento con nuevos hiperparámetros, o ambos. El modelo más reciente puede entonces compararse con el modelo existente utilizando una forma de pruebas A/B (véase 6.1.9).

El flujo de trabajo de ML mostrado en la Figura 1 es una secuencia lógica; sin embargo, en la práctica, el flujo de trabajo se aplica de forma iterativa, con pasos que se repiten. Los pasos mostrados en la Figura 1 no incluyen la integración del modelo de ML con las partes no basadas en ML del sistema general. Típicamente, los modelos de ML no pueden desplegarse de forma aislada y deben integrarse con componentes no basados en ML (por ejemplo, en aplicaciones de visión, se utiliza un pipeline de datos para limpiar y modificar los datos antes de enviarlos al modelo de ML). Cuando el modelo es parte de un sistema mayor, debe integrarse en este sistema antes del despliegue. En este caso, pueden realizarse pruebas a nivel de integración, sistema y aceptación.

Figura 1: Flujo de Trabajo de ML

3.1.3 Ejercicio Práctico: Crear un Modelo de Machine Learning

Seleccionar, entrenar y probar un modelo de clasificación utilizando aprendizaje supervisado. Explicar la diferencia entre evaluación/ajuste y pruebas comparando la precisión obtenida con los conjuntos de datos de validación y prueba.

3.1.4 Modelos Preentrenados, Ajuste Fino (Fine-Tuning) y Generación Aumentada por Recuperación (RAG)

Entrenar un nuevo modelo de IA desde cero es costoso y requiere mucho tiempo. Para abordar esto, una solución común es el ajuste fino (fine-tuning), que consiste en tomar una red neuronal preentrenada y adaptarla para realizar una tarea nueva y diferente. Uno de los beneficios clave es que

requiere muchos menos datos de entrenamiento (y esfuerzo de entrenamiento) en comparación con la construcción de un modelo desde cero.

El modelo preentrenado se ajusta finamente realizando entrenamiento adicional con datos específicos de la nueva tarea. El ajuste puede aplicarse a toda la red neuronal, solo a capas específicas (típicamente cerca del extremo de salida de la red neuronal) o a capas adicionales. Después del entrenamiento, se evalúa el rendimiento funcional de ML del modelo y, basándose en estos resultados, puede realizarse un ajuste fino adicional hasta que el modelo cumpla con los criterios de aceptación necesarios.

El éxito del ajuste fino depende de la similitud entre la tarea original y la nueva. Las diferencias pequeñas pueden conducir a un ajuste fino muy efectivo. Por ejemplo, adaptar un clasificador de imágenes de razas de gatos para identificar razas de perros probablemente funcionará bien. Sin embargo, adaptarlo para acentos hablados es menos efectivo debido a la mayor diferencia. De manera similar, el ajuste fino de un LLM para el análisis de valores límite definido por ISTQB requiere un cambio pequeño para el LLM y es fácilmente alcanzable con buenos datos de entrenamiento.

Una alternativa al ajuste fino es la Generación Aumentada por Recuperación (RAG), que consiste en proporcionar fuentes de datos al LLM que son específicas para la tarea requerida. Estas fuentes de datos se transforman en un formato consultable que permite compararlas con el tema del prompt. Una vez que se identifican documentos relevantes, estos se incorporan en un prompt mejorado que se pasa al LLM. Como ahora se proporciona más información pertinente al LLM, su respuesta correspondiente probablemente será más precisa. Con RAG, no se realiza ningún cambio en el modelo preentrenado.

Un modelo preentrenado puede utilizar RAG, ajuste fino, o ambos conjuntamente para mejorar el rendimiento. Típicamente, cualquier sesgo o vulnerabilidad del modelo preentrenado se trasladará al nuevo modelo, por lo que es

necesario realizar pruebas para confirmar que funciona de manera fiable y equitativa en la nueva tarea.

3.2 Datos para Machine Learning

La preparación de datos es reconocida como una de las actividades más cruciales e intensivas en recursos del flujo de trabajo de ML. Si los datos operativos difieren significativamente de los datos de entrenamiento, las suposiciones sobre el rendimiento funcional de ML y la seguridad pueden dejar de ser válidas. La preparación de datos típicamente consume una proporción significativamente mayor del esfuerzo total en comparación con otras etapas, como la selección y construcción del modelo. La preparación de datos está intrínsecamente vinculada al pipeline de datos, que procesa los datos en bruto y los transforma en un formato utilizable tanto para el entrenamiento como para la predicción por parte de los modelos de ML.

3.2.1 Actividades en la Preparación de Datos

La preparación de datos apoya el logro de la calidad e idoneidad de los datos para el entrenamiento del modelo. Involucra varias actividades clave:

- Adquisición de datos:
- Identificación de tipos de datos relevantes (por ejemplo, numéricos, categóricos, imágenes, texto).
- Recopilación de datos de diversas fuentes, como bases de datos, APIs o sensores en tiempo real.
- Etiquetado de datos para tareas de aprendizaje supervisado, verificando la precisión y consistencia. Los datos adquiridos pueden adoptar diversas formas (por ejemplo, numéricos, categóricos, imágenes, tabulares, texto, series temporales, sensores, geoespaciales, vídeo y audio).
- Preprocesamiento de datos:
- Limpieza de datos, incluyendo: eliminación de defectos, duplicados y valores atípicos para verificar

la precisión y consistencia de los datos; imputación de valores faltantes utilizando técnicas como la media, la mediana o la moda para mantener la completitud de los datos; anonimización o eliminación de información personal para proteger la privacidad y cumplir con las regulaciones.

- Transformación de formatos de datos, escalado y normalización para lograr consistencia.
- Aumento de datos para incrementar el tamaño de la muestra, incorporación de ejemplos adversariales para mejorar la robustez contra ataques adversariales y generación de datos sintéticos.
- Muestreo de subconjuntos para reducir los tiempos de entrenamiento y los costos computacionales.
- Ingeniería de características:
 - Selección de características relevantes basándose en su contribución al rendimiento del modelo de ML.
 - Extracción de un subconjunto de características informativas y no redundantes a partir de las características existentes para reducir los tiempos de entrenamiento y los costos computacionales.

En paralelo a estas actividades de preparación de datos, también se lleva a cabo típicamente un análisis exploratorio de datos (EDA) para proporcionar información sobre los datos. Esto incluye: - descubrir tendencias, patrones y anomalías en los datos; - visualizar datos mediante gráficos y diagramas para una mejor comprensión.

La preparación de datos de entrenamiento es típicamente un proceso iterativo, a menudo realizado manualmente, y las actividades individuales de preparación de datos pueden reordenarse u omitirse según los requisitos específicos del proyecto. Los datos operativos deben coincidir con las características de los datos de entrenamiento (por ejemplo, distribuciones de datos y rangos de características) para que el modelo funcione como se espera en producción. Sin embargo, los pasos de preparación en sí pueden ajustarse para eficiencia y escalabilidad en producción.

3.2.2 Ejercicio Práctico: Preparación de Datos como Apoyo a la Creación de un Modelo de Machine Learning

Para un conjunto de datos dado, realizar los pasos de preparación de datos aplicables como se describe en la Sección 3.2.1 para producir un conjunto de datos que se utilizará para crear un modelo de clasificación mediante aprendizaje supervisado.

3.2.3 Conjuntos de Datos de Entrenamiento, Validación y Prueba

Lógicamente, se requieren tres conjuntos de datos equivalentes (por ejemplo, seleccionados aleatoriamente de un único conjunto de datos representativo) para desarrollar un modelo de ML: - Un conjunto de datos de entrenamiento se utiliza para entrenar el modelo. - Un conjunto de datos de validación se utiliza para evaluar y posteriormente ajustar el modelo. - Un conjunto de datos de prueba, también conocido como conjunto de datos de reserva (holdout), se utiliza para probar el modelo ajustado.

Si hay abundancia de datos adecuados, la cantidad de datos utilizada en el flujo de trabajo de ML para entrenamiento, evaluación y pruebas típicamente depende de los siguientes factores: - La complejidad esperada del modelo. - El algoritmo utilizado para entrenar el modelo. - La disponibilidad de recursos, como RAM, espacio en disco, potencia de cómputo, ancho de banda de red y el tiempo disponible. - La confianza deseada en el modelo resultante.

Cuando los datos son limitados, una división en tres partes (entrenamiento, validación y prueba) puede resultar en datos insuficientes para un entrenamiento efectivo del modelo, aumentando así el riesgo de subajuste (underfitting). Para abordar esto, una estrategia común es reservar un pequeño conjunto de prueba final de reserva (hold-out) si es factible. Los datos restantes (el conjunto combinado de entrenamiento y validación) se utilizan entonces para técnicas como la validación cruzada de k pliegues (k -fold cross-validation) (donde k es un entero especificado por el usuario, comúnmente 5 o 10).

En la validación cruzada de k pliegues, estos datos se dividen en k 'pliegues' (folds). Para cada pliegue, el modelo se entrena con $k-1$ pliegues y se valida con el pliegue reservado. Este proceso se repite k veces, con cada pliegue sirviendo como conjunto de validación una vez. Esto permite un ajuste robusto de hiperparámetros y una estimación del rendimiento funcional de ML. Los datos se asignan típicamente de forma aleatoria a los pliegues, a menudo utilizando muestreo estratificado para hacer que cada pliegue sea representativo, especialmente con datos desequilibrados o conjuntos de datos pequeños.

Las métricas de rendimiento (por ejemplo, accuracy, F1-score – véase 3.3.1) de la validación de cada pliegue se promedian entonces para proporcionar una estimación más fiable de la capacidad de generalización del modelo. Después de identificar los hiperparámetros óptimos mediante la validación cruzada, típicamente se entrena un modelo final con todo el conjunto de entrenamiento y validación (todos los datos excepto el conjunto de prueba de reserva) utilizando estos hiperparámetros. Este modelo final se evalúa entonces una vez con el conjunto de prueba de reserva para una evaluación final e imparcial del rendimiento. Si un conjunto de prueba de reserva no es factible debido a una escasez extrema de datos, el rendimiento promedio de la validación cruzada está sesgado de manera optimista y no puede servir como una estimación imparcial.

Otros métodos de remuestreo para datos limitados incluyen la validación cruzada dejando uno fuera (leave-one-out cross-validation, un caso especial de k -fold donde k es igual al número de muestras) y técnicas de bootstrap.

3.3 Métricas de Rendimiento Funcional de ML para Clasificación

En tareas de clasificación (véase 3.1.1), se puede utilizar una matriz de confusión para evaluar las predicciones de un modelo, categorizándolas como verdaderos positivos, verdaderos negativos, falsos positivos o falsos negativos. De esta se derivan métricas clave como accuracy, precision, recall y F1-score. Estas métricas miden la calidad de la

clasificación, destacando las fortalezas y debilidades del modelo de ML. Esta sección explora el cálculo y la interpretación de estas métricas para evaluar el rendimiento funcional de ML.

3.3.1 Cálculo de Métricas de Rendimiento Funcional de Machine Learning

En un problema de clasificación, un modelo rara vez predecirá los resultados correctamente todo el tiempo, en parte debido a la naturaleza probabilística de los modelos de ML y al ruido en los datos. Para cualquier problema de este tipo, se puede crear una matriz de confusión con las siguientes posibilidades:

	Positivo Real	Negativo Real
Predicción Positiva	Verdadero Positivo (TP)	Falso Positivo (FP)
Predicción Negativa	Falso Negativo (FN)	Verdadero Negativo (TN)

Figura 2: Matriz de Confusión

Nótese que la matriz de confusión mostrada en la Figura 2 puede presentarse de forma diferente (por ejemplo, con predicción y real intercambiados), pero siempre proporcionará valores para las cuatro situaciones posibles de verdadero positivo (TP), verdadero negativo (TN), falso positivo (FP) y falso negativo (FN).

Basándose en la matriz de confusión, se definen las siguientes métricas:

- Accuracy = $(TP + TN) / (TP + TN + FP + FN) * 100\%$ Accuracy mide el porcentaje de todas las clasificaciones correctas.
- Precision = $TP / (TP + FP) * 100\%$ Precision mide la proporción de positivos que fueron predichos correctamente. Es una medida de cuán seguro se puede estar acerca de las predicciones positivas.
- Recall = $TP / (TP + FN) * 100\%$ Recall (también conocido como sensibilidad) mide la proporción de positivos reales que se predicen correctamente. Es una medida de cuán seguro se puede estar de que no se omitirá ningún positivo.
- F1-score = $2 * (Precision * Recall) / (Precision + Recall)$ El F1-score se

calcula como la media armónica de precision y recall, con valores que van de 0 a 100. Una puntuación cercana a 100 significa que el modelo logra tanto alta precision como alto recall, indicando que los errores de clasificación (falsos positivos y falsos negativos) tienen un impacto mínimo. Por el contrario, un F1-score bajo indica que el modelo tiene dificultades para identificar positivos con precisión, ya sea omitiendo casos verdaderos o generando muchas falsas alarmas.

3.3.2 Ejercicio Práctico: Evaluar un Modelo de Machine Learning utilizando Métricas de Rendimiento Funcional de ML Seleccionadas

Utilizando el modelo de clasificación entrenado en el ejercicio anterior, calcular y mostrar los valores de accuracy, precision, recall y F1-score. Cuando sea aplicable, utilizar las funciones de biblioteca proporcionadas por su framework de desarrollo de ML para realizar los cálculos.

3.3.3 Ejercicio Práctico: Mostrar el Impacto de Diferentes Combinaciones de Modelos de Machine Learning y Conjuntos de Datos

Siguiendo el ejercicio anterior, utilizar diferentes combinaciones de modelos de ML y conjuntos de datos para observar su efecto en el entrenamiento del modelo de ML, así como en el comportamiento final del modelo. Observar los tiempos de entrenamiento y las métricas de rendimiento funcional de ML.

3.4 Redes Neuronales

Las redes neuronales artificiales fueron diseñadas inicialmente para imitar el funcionamiento del cerebro humano, que puede considerarse como una red de neuronas biológicas interconectadas.

El perceptrón de capa única es uno de los primeros ejemplos de implementación de una red neuronal artificial, compuesto por una sola capa. Puede utilizarse para el aprendizaje supervisado de clasificadores binarios para problemas linealmente separables, que determinan si una entrada

pertenece a una clase específica o no. Por ejemplo, un perceptrón puede distinguir entre correos electrónicos que son spam y los que no lo son, aprendiendo a separar las características de las dos categorías con una línea recta en el espacio de entrada.

La mayoría de las redes neuronales actuales se consideran redes neuronales profundas (deep neural networks) porque están compuestas por varias capas. Las redes completamente conectadas pueden considerarse como perceptrones multicapa (véase la Figura 3).

Figura 3: Estructura de una red neuronal profunda

3.4.1 Estructura y Funcionamiento de una Red Neuronal Profunda

Una red neuronal profunda se describe típicamente como compuesta por tres tipos principales de capas. La capa de entrada recibe las entradas, por ejemplo, valores de píxeles de una cámara. La capa de salida proporciona resultados al mundo exterior. Esto podría ser, por ejemplo, un valor que indica la probabilidad de que la imagen de entrada sea de un gato. Entre las capas de entrada y salida se encuentran las capas ocultas compuestas por neuronas artificiales, también conocidas como nodos. En muchas arquitecturas comunes, como las redes completamente conectadas, las neuronas de una capa están conectadas a cada una de las neuronas de la capa siguiente, y puede haber diferentes números de neuronas en cada capa sucesiva.

Las neuronas realizan cálculos y transmiten información a través de la red desde las neuronas de entrada hasta las neuronas de salida, transformando gradualmente los datos de entrada en representaciones cada vez más abstractas hasta alcanzar la salida.

El cálculo realizado por cada neurona (después de las de la capa de entrada) genera lo que se conoce como el valor de activación. Este valor se calcula primero computando una suma ponderada de los valores de activación de todas las neuronas conectadas en la capa anterior, donde cada conexión tiene su propio peso independiente, y sumando el sesgo (bias) individual de la neurona. Esta suma se pasa

luego a través de una fórmula no lineal llamada función de activación. Nótese que este sesgo (bias) no está relacionado con el sesgo considerado en 5.1.2. El uso de diferentes funciones de activación produce diferentes valores de activación.

Figura 4: Cálculo realizado por cada neurona

Los pesos que conectan las neuronas y el valor de sesgo (bias) de cada neurona se inicializan típicamente con valores aleatorios pequeños (los sesgos a veces con cero) al inicio del entrenamiento. Los datos de entrenamiento se pasan a través de la red, con cada neurona ejecutando la función de activación, para generar una salida en última instancia. La salida generada se compara entonces con el resultado correcto conocido. El error (o pérdida) resultante, que cuantifica esta diferencia, se retroalimenta a través de la red para ajustar los valores de los pesos y sesgos, minimizando así esta diferencia. A medida que más datos de entrenamiento se alimentan a través de la red (cada pasada a través del conjunto de datos de entrenamiento se denomina una época o epoch), los valores de los pesos y sesgos se ajustan gradualmente mientras la red aprende. Después de cierto tiempo, idealmente, la salida producida se considera suficientemente buena para finalizar el entrenamiento.

3.4.2 Ejercicio Práctico: Experimentar la Implementación de un Perceptrón

Los estudiantes serán guiados a través de un ejercicio que demuestra cómo un Perceptrón aprende una función simple, como una función AND. El ejercicio debe cubrir cómo un Perceptrón aprende modificando sus pesos y valores de sesgo (bias) a lo largo de múltiples épocas (epochs) hasta que el error se minimiza a cero. Pueden utilizarse diversos mecanismos (por ejemplo, hoja de cálculo, simulación) para esta actividad.

3.4.3 Medidas de Cobertura para Redes Neuronales

Las métricas de cobertura estructural de redes neuronales han surgido para evaluar cuán exhaustivamente las entradas

de prueba ejercitan los mecanismos internos de un modelo. Como las redes neuronales no siguen caminos explícitamente codificados, sino que su comportamiento está dictado por los pesos aprendidos, los valores de sesgo (bias) y las activaciones, esto crea la necesidad de medidas de cobertura especializadas para evaluar en qué medida diferentes partes de la red han sido activadas bajo condiciones de prueba.

Los enfoques típicos incluyen [COV_REF]: - Cobertura de Neuronas (Neuron Coverage): Mide la proporción de neuronas en la red cuya salida excede un umbral especificado durante las pruebas. - Cobertura de Neuronas por k-Multisección (k-Multisection Neuron Coverage, kMNC): El rango de salida posible de cada neurona se divide en k secciones. kMNC es la proporción de estas secciones activadas durante las pruebas. - Cobertura de Límites de Neuronas (Neuron Boundary Coverage, NBC): Mide la proporción de neuronas en la red cuya salida excede el máximo alcanzado durante el entrenamiento o es inferior al mínimo alcanzado durante el entrenamiento en las pruebas.

Estas métricas de cobertura pueden ser útiles en las pruebas de IA, principalmente al revelar áreas del modelo que permanecen sin probar, las cuales pueden potencialmente ocultar defectos o comportamientos no aprendidos. Por ejemplo, si ciertas neuronas o capas nunca se activan durante las pruebas, el rendimiento del modelo en las fronteras de decisión relacionadas podría ser cuestionable. Tales perspectivas ayudan a los testers a diseñar entradas de prueba o condiciones de prueba adicionales para ejercitar partes poco exploradas de la red. En la actualidad, las herramientas comerciales que soportan estas medidas de cobertura específicas son limitadas.

Sin embargo, la cobertura estructural por sí sola no garantiza que una red neuronal generalice bien o maneje variaciones del mundo real. Las redes neuronales pueden aprender correlaciones espurias, lo que lleva a activaciones correctas por razones incorrectas. En consecuencia, los testers también deben utilizar otras técnicas de prueba, como las pruebas adversariales y las pruebas metamórficas, como se describe en los capítulos 5, 6 y 7.

4 Pruebas de Sistemas Basados en IA - 195 minutos

Palabras clave: Attack, exploratory testing, risk-based testing, test oracle

Palabras clave específicas de IA: Adaptive AI-based system, AI-based system, generative AI, large language model, locked AI-based system

Objetivos de Aprendizaje para el Capítulo 4:

4.1 Introducción a las Pruebas de Sistemas Basados en IA - AI-4.1.1 (K2) Comparar la testabilidad de los sistemas basados en IA bloqueados y adaptativos - AI-4.1.2 (K2) Explicar por qué a menudo se necesita un enfoque estadístico al probar sistemas basados en IA - AI-4.1.3 (K2) Explicar los desafíos y soluciones relacionados con los oráculos de prueba para sistemas basados en IA

4.2 Pruebas de IA Generativa y LLM - AI-4.2.1 (K2) Explicar cómo se puede probar la IA generativa - AI-4.2.2 (K3) Implementar red teaming para sistemas GenAI - HO-4.2.3 (H2) Aplicar pruebas exploratorias a un LLM que realiza análisis de valores límite

4.3 Niveles de Prueba y Sistemas de Machine Learning - AI-4.3.1 (K2) Resumir los niveles de prueba utilizados para desarrollar sistemas de machine learning - AI-4.3.2 (K2) Explicar cómo se aplican las pruebas basadas en riesgos a los sistemas de machine learning

4.1 Introducción a las Pruebas de Sistemas Basados en IA

Las pruebas de sistemas basados en IA presentan desafíos únicos en comparación con las pruebas de software convencional. En primer lugar, los sistemas basados en IA pueden clasificarse ampliamente como bloqueados o adaptativos. Los sistemas basados en IA bloqueados, con comportamiento fijo después del despliegue, son más fáciles de probar debido a su naturaleza mayormente determinista,

mientras que los sistemas adaptativos, que evolucionan y aprenden, introducen complejidad ya que su comportamiento puede cambiar de manera impredecible. Además, la naturaleza probabilística de muchos modelos de IA a menudo requiere pruebas estadísticas, ya que los métodos deterministas pueden ser insuficientes para evaluar resultados influenciados por datos y probabilidades. Esto requiere la evaluación de distribuciones de rendimiento y niveles de confianza en diversos escenarios.

Un desafío central en las pruebas, conocido como el 'problema del oráculo de prueba', surge al determinar si la salida producida por el sistema bajo prueba es correcta para una entrada dada. En el software tradicional, las especificaciones bien definidas hacen relativamente sencillo especificar resultados esperados y verificar la corrección. Sin embargo, con los sistemas basados en IA, especialmente aquellos que abordan tareas complejas o subjetivas, definir claramente los resultados esperados puede ser difícil o incluso imposible. Esta incertidumbre se agrava para tareas que exceden las capacidades humanas, involucran ambigüedad o carecen de una 'verdad fundamental' objetiva, lo que dificulta implementar oráculos de prueba automatizados y a veces requiere juicio experto o razonamiento estadístico.

Los requisitos del sistema vagos o incompletos exacerban aún más el problema del oráculo de prueba en las pruebas de IA. Las soluciones pueden incluir el uso de evaluaciones estadísticas, la consulta a expertos del dominio o la definición de una 'verdad fundamental' contra la cual evaluar las salidas. Estos enfoques buscan establecer expectativas confiables y evaluar eficazmente los sistemas basados en IA.

4.1.1 Sistemas Basados en IA Bloqueados y Adaptativos

Muchos sistemas basados en IA en uso hoy en día son sistemas basados en IA bloqueados, que no cambian su comportamiento una vez desplegados. Un ejemplo de un sistema basado en IA bloqueado es un modelo ML desplegado basado en una DNN, donde los pesos y sesgos de la DNN se fijan después del desarrollo y solo pueden

modificarse si la DNN se reentrena. La tecnología de conducción autónoma relacionada con la seguridad a menudo depende de sistemas basados en IA bloqueados para tareas específicas, como la detección de carriles o el reconocimiento de señales de tráfico.

En contraste, un sistema basado en IA adaptativo, como un sistema de aprendizaje por refuerzo, puede adaptar su comportamiento una vez desplegado. Los cambios pueden basarse en una función de recompensa o en la adaptación a un nuevo entorno operativo, pero los detalles específicos de tales cambios no pueden predecirse con anticipación. Por ejemplo, una plataforma de comercio electrónico podría usar IA adaptativa para recomendar productos a los usuarios basándose en su comportamiento pasado y preferencias en evolución.

Muchos sistemas GenAI, como los chatbots basados en LLM, se despliegan como modelos bloqueados en tiempo de ejecución pero se actualizan periódicamente, ubicándolos entre sistemas completamente bloqueados y completamente adaptativos.

En la práctica, los sistemas basados en IA abarcan un continuo desde sistemas completamente deterministas y bloqueados que producen la misma salida para una entrada dada, hasta sistemas deliberadamente no deterministas y de autoaprendizaje que adaptan y evolucionan su comportamiento.

Los sistemas basados en IA bloqueados son mucho más fáciles de probar que los sistemas basados en IA adaptativos, ya que son en gran medida deterministas y, por lo tanto, los resultados esperados no cambian. Un sistema basado en IA bloqueado actualizado se considera típicamente un nuevo sistema, y se requiere una nueva ronda de pruebas. Nótese, sin embargo, que aunque los sistemas basados en IA bloqueados se consideran generalmente deterministas, en la práctica, las redes neuronales grandes pueden exhibir comportamiento no determinista debido a factores como los límites de precisión de punto flotante y variaciones en la ejecución del hardware, particularmente al usar computación paralela o GPUs.

Un sistema basado en IA adaptativo puede probarse rigurosamente antes del despliegue (por ejemplo, simulando cambios en las condiciones ambientales, probando el mecanismo de aprendizaje en sí, o probando su capacidad para adaptarse apropiadamente en escenarios controlados). Sin embargo, estas pruebas son más complejas que para un sistema basado en IA bloqueado, ya que un sistema adaptativo puede cambiar su comportamiento durante las pruebas o como resultado de las pruebas.

Como los nuevos comportamientos de un sistema basado en IA adaptativo no siempre pueden predecirse, tales comportamientos impredecibles no pueden probarse por adelantado, ni pueden prepararse casos de prueba para ellos. Se puede construir y ejecutar un conjunto de pruebas automatizadas, centrado en la funcionalidad central del sistema, cada vez que el sistema experimente cambios significativos para verificar que las adaptaciones son seguras.

Las pruebas también pueden usarse para verificar que el rendimiento del sistema no se ha degradado más allá de un cierto umbral. Estas pruebas pueden ser en respuesta a cambios significativos del sistema o como parte del monitoreo continuo.

4.1.2 Justificación de un Enfoque Estadístico para las Pruebas de Sistemas Basados en IA

Las pruebas de sistemas basados en IA presentan desafíos únicos debido a su naturaleza basada en datos y probabilística. Las razones por las que se necesita un enfoque estadístico para probar sistemas basados en IA incluyen:

- No Determinismo - Los sistemas basados en IA son fundamentalmente probabilísticos y, por lo tanto, a menudo exhiben comportamiento no determinista, lo que significa que las mismas entradas no siempre producen la misma salida. Esto puede deberse a elementos estocásticos en su arquitectura o a su implementación de mapeos probabilísticos aprendidos de los datos de entrenamiento. En

consecuencia, una sola prueba, como una instancia en la que un modelo clasifica incorrectamente un gato como un perro, no puede reflejar con precisión la corrección funcional general del modelo de IA. Para confirmar que los resultados de las pruebas superan de manera confiable esta incertidumbre, el conjunto de pruebas debe ser suficientemente grande para proporcionar resultados estadísticamente significativos.

- **Evaluación de Rendimiento Distribucional** - Los modelos de IA se entrenan con distribuciones de datos específicas que no coinciden exactamente con su entorno operativo. Para evaluar cómo se desempeñará un modelo en condiciones del mundo real, se debe probar una muestra estadísticamente significativa de escenarios de distribuciones de datos operativos relevantes. Esto confirma que las pruebas capturan la variabilidad operativa en los datos y los comportamientos subsecuentes del modelo.
- **Manejo de Incertidumbre y Sesgo** - Los sistemas basados en IA son susceptibles a sesgos en los datos y pueden producir predicciones confiadas pero incorrectas. Las pruebas estadísticas permiten a los profesionales cuantificar y analizar la precisión del modelo, la equidad y la robustez de IA a través de métricas de rendimiento como intervalos de confianza, pruebas de hipótesis y análisis de errores.
- **Contexto Regulatorio y de Seguridad** - En industrias reguladas (por ejemplo, salud, transporte), a menudo se requiere demostrar que un sistema basado en IA cumple con los umbrales de seguridad o equidad con alta confianza. Los métodos estadísticos respaldan las afirmaciones sobre la confiabilidad de un modelo en una amplia gama de escenarios, en lugar de solo para ejemplos específicos.

Un enfoque estadístico para las pruebas de MLS probabilísticos se proporciona en 6.1.3.

4.1.3 Oráculos de Prueba para Sistemas Basados en IA

Las pruebas de sistemas basados en IA pueden ser desafiantes, particularmente al determinar los resultados esperados (el problema del oráculo de prueba). Estas dificultades surgen de varios factores inherentes a la IA:

- **Naturaleza Probabilística y No Determinista:** Las salidas de IA pueden variar incluso con entradas idénticas. Mientras que muchos sistemas (por ejemplo, en aprendizaje supervisado) tienen un único valor objetivo correcto, las salidas del modelo son probabilísticas y definir un oráculo estricto de aprobado/fallido a menudo requiere establecer umbrales o rangos de tolerancia.
- **Desarrollo Exploratorio y Especificaciones Incompletas:** El desarrollo de IA es frecuentemente exploratorio. Los requisitos del sistema pueden evolucionar, estar incompletos o simplemente faltar, careciendo de las especificaciones detalladas necesarias para generar resultados esperados precisos.
- **Complejidad de las Tareas:** Los sistemas basados en IA a menudo abordan tareas demasiado complejas para la verificación humana directa, haciendo impracticables las verificaciones manuales de resultados esperados.
- **Subjetividad del Comportamiento:** La corrección del comportamiento de un sistema basado en IA puede ser subjetiva. Por ejemplo, las expectativas de los usuarios para asistentes virtuales pueden variar ampliamente, complicando el establecimiento de resultados esperados universalmente aceptados.
- **Sistemas de Autoaprendizaje:** Estos sistemas basados en IA actualizan continuamente sus modelos internos basándose en nuevos datos encontrados después del despliegue. Esto hace que el comportamiento 'correcto' del sistema basado en IA cambie con el tiempo para que las respuestas del sistema permanezcan efectivas y apropiadas incluso

a medida que la definición de 'correcto' evoluciona con el tiempo. Como resultado, un conjunto inicial de resultados esperados puede invalidarse rápidamente.

El problema del oráculo de prueba con sistemas basados en IA puede abordarse de varias maneras:

- Definición de Límites de Salida: Los probadores pueden usar rangos aceptables acordados, distribuciones, límites especificados y tolerancias, como un automóvil autónomo que se detiene dentro de una distancia máxima.
- Definición de Límites Ambientales: Los probadores deben especificar valores para las condiciones del entorno de prueba (por ejemplo, niveles de iluminación, temperatura, latencia de red) para garantizar que las salidas sean predecibles y repetibles.
- Consulta a Expertos: Los expertos del dominio pueden ayudar a definir los resultados esperados, aunque sus opiniones pueden diferir o ser falibles.
- Pruebas Especializadas: Las pruebas A/B, las pruebas back-to-back y las pruebas metamórficas, entre otras, pueden evaluar sistemas basados en IA comparando comportamientos o verificando propiedades, a menudo sin requerir resultados esperados explícitos para cada caso.
- Oráculos Proxy: Usar sistemas o modelos secundarios (incluyendo otros sistemas de IA) para evaluar o validar salidas cuando los resultados esperados directos no están disponibles, como entrenar un modelo proxy con datos etiquetados para predecir en pruebas no etiquetadas.

4.2 Pruebas de IA Generativa y Modelos de Lenguaje Grande

Esta sección describe métodos prácticos para validar sistemas GenAI, cubriendo la evaluación de caja negra, red teaming y técnicas prácticas. Destaca los desafíos de las diversas entradas, parámetros ajustables y ventanas de contexto, y aborda tanto las medidas de calidad funcionales como no funcionales utilizando benchmarks y ejercicios dirigidos. Consulte el syllabus CT-GenAI [CT-GenAI] para más detalles sobre el uso de sistemas GenAI para pruebas.

4.2.1 Pruebas de IA Generativa

Las pruebas de GenAI implican evaluar la corrección, coherencia y creatividad de sus salidas, incluyendo la originalidad y novedad de los resultados generados, y verificar que se cumplan los requisitos especificados, tanto funcionales como no funcionales. Dado que los sistemas GenAI pueden producir texto, imágenes, video y audio, las estrategias de prueba deben adaptarse para evaluar las características de calidad de dicho contenido.

Un enfoque común es la prueba de caja negra, donde los probadores alimentan diversas entradas (prompts, imágenes, datos parciales) al sistema y evalúan los resultados de las pruebas (véase red teaming, 4.2.2). Se consideran factores como la claridad, la originalidad y la adherencia a reglas específicas del dominio. Este enfoque, ya sea manual o automatizado, es particularmente útil para aplicaciones de usuario final, como chatbots o herramientas de diseño, donde la satisfacción del usuario depende de la utilidad y plausibilidad del contenido generado.

Un desafío significativo en las pruebas de un modelo GenAI es el 'problema de la explosión de entradas', en el cual las entradas pueden ser altamente diversas y complejas de controlar. Por ejemplo, hay prompts de sistema opcionales y un prompt de usuario, que pueden incluir enormes cantidades de datos dispares, y GenAI también puede accederse vía una API. También hay múltiples parámetros a considerar, como la temperatura y los tokens máximos, que también afectarán la salida. Además, la ventana de contexto, que retiene partes anteriores de una conversación, también afecta la salida generada.

En muchas situaciones, evaluar la salida puede involucrar revisión manual; sin embargo, determinar si una prueba ha pasado o fallado depende de criterios de evaluación cualitativos definidos en los requisitos. Alternativamente, un segundo sistema GenAI puede usarse a menudo para determinar automáticamente el resultado de la prueba. Por ejemplo, la corrección de una imagen generada por un sistema GenAI puede verificarse mediante un sistema de reconocimiento de imágenes. Tales enfoques deben usarse

con cuidado, ya que pueden reproducir sesgos o errores similares.

Las pruebas no funcionales pueden ser tan necesarias como las pruebas funcionales para sistemas GenAI. Esto incluye evaluar la utilización de recursos durante tanto la inferencia como el entrenamiento para verificar la operación eficiente y la relación costo-efectividad. Las medidas incluyen el uso de CPU y GPU, consumo de memoria, ancho de banda de red y tiempos de respuesta.

Los conjuntos de benchmarks proporcionan marcos de evaluación estandarizados para evaluar las capacidades de GenAI. Estos conjuntos de datos curados y tareas asociadas permiten la comparación consistente entre diferentes modelos, midiendo diversos aspectos, desde la comprensión del lenguaje hasta el razonamiento y las capacidades de codificación. Al medir GenAI contra estos benchmarks, se pueden identificar áreas de mejora de manera sistemática y reproducible.

4.2.2 Red Teaming

El Red Teaming (RT) es una forma sistemática, a menudo de caja negra, de ataque de fallas que sondea un sistema basado en IA para identificar capacidades dañinas, particularmente en sus salidas. Inspirándose en prácticas históricas como los juegos de guerra militares y los tiger teams de la NASA, el RT de IA implica 'atacar' un sistema basado en IA, con el objetivo de causar deliberadamente que el sistema produzca resultados dañinos o indeseables, como violaciones de privacidad, la expresión de opiniones racistas o proporcionar orientación para realizar ataques químicos, biológicos, radiológicos o nucleares (CBRN). Una vez identificadas tales capacidades, se utilizan para actualizar y fortalecer el sistema, haciéndolo menos propenso a tales salidas en el futuro. Aunque el RT puede aplicarse a cualquier sistema basado en IA, es especialmente crítico para GenAI debido al amplio rango de posibles entradas y salidas, que crean un extenso 'espacio de ataque'. El RT típicamente cubre el sistema basado en IA completo de extremo a extremo, pero puede aplicarse solo al modelo de IA.

Muchas organizaciones enfocan el RT en vulnerabilidades de seguridad y protección; sin embargo, también puede usarse para detectar capacidades dañinas en otras áreas, como confiabilidad, privacidad, equidad, sesgo y la generación de desinformación. El RT se está convirtiendo cada vez más en una expectativa regulatoria para algunos tipos de sistemas basados en IA, como se ve en marcos como el EU AI Act [EU AI Act]. Como un enfoque de evaluación adaptativo y dinámico en el que los prompts pueden actualizarse inmediatamente en respuesta a las salidas, el RT complementa los enfoques estáticos como el benchmarking al probar sistemas bajo condiciones extremas e inesperadas.

Cuando el RT se usa para evaluaciones de seguridad, el sistema se prueba para identificar vulnerabilidades a ataques externos, incluyendo tanto factores de seguridad no específicos de IA como específicos de IA, como ataques de inyección indirecta de prompts y la ocultación de contenido malicioso en documentos utilizados por RAG. Mientras que el RT de seguridad tiende a enfocarse en entradas maliciosas, para evaluaciones de protección y otras, el objetivo a menudo es identificar cómo el sistema podría crear salidas dañinas bajo uso ordinario, como generar consejos médicos inseguros, sin ninguna intención adversarial del usuario.

El RT es más efectivo cuando se realiza antes de que un sistema sea desplegado, pero después de que las evaluaciones internas iniciales de calidad estén completas. La actividad central es a menudo el prompting interactivo, donde los red teamers participan en diálogos de múltiples turnos (por ejemplo, 15-20 turnos) para provocar comportamientos defectuosos o que violen políticas. El enfoque típicamente involucra: 1) Reunir un equipo diverso de probadores para cubrir una amplia gama de perspectivas y vectores de ataque. 2) Proporcionar acceso al sistema basado en IA en un entorno de prueba seguro. 3) Provocar al sistema para identificar vulnerabilidades, a través de exploración abierta o usando listas de verificación. 4) Analizar las fallas identificadas para comprender las amenazas. 5) Crear conjuntos de datos a partir de estas amenazas para apoyar las mitigaciones y mejoras del sistema.

Para lograr una cobertura integral mediante RT, las organizaciones emplean varias estrategias más allá de pequeños equipos de expertos. Estas incluyen enfoques manuales como la generación de prompts mediante crowdsourcing, y enfoques automatizados, como donde se usa un LLM para generar numerosos prompts de ataque, y las salidas son luego verificadas por otro LLM. Los enfoques híbridos combinan la creatividad de los probadores humanos con la escalabilidad de la automatización.

El RT, que es proactivo y enfocado en el pre-despliegue, complementa al Blue Teaming, que implica monitoreo defensivo continuo en tiempo real y filtrado de entradas a un sistema basado en IA operativo para protegerlo contra ataques. Los conocimientos del RT pueden usarse para mejorar el monitoreo y los filtros utilizados en el Blue Teaming.

4.2.3 Ejercicio Práctico: Pruebas Exploratorias de un Modelo de Lenguaje Grande

Los estudiantes realizarán pruebas exploratorias de un LLM. Se les proporcionará a los estudiantes una hoja de sesión de pruebas exploratorias enfocada en probar la capacidad del LLM para generar casos de prueba utilizando análisis de valores límite de 2 valores y 3 valores. Como parte del debriefing de la sesión, verificarán la corrección y completitud de la salida.

4.3 Niveles de Prueba y Sistemas de Machine Learning

Los MLS requieren niveles de prueba especializados para abordar riesgos únicos de ML. Estos son las pruebas de datos de entrada y las pruebas de modelo ML. Además, los niveles de prueba convencionales siguen siendo relevantes, incluyendo pruebas de componente, integración de componentes, sistema y aceptación.

4.3.1 Niveles de Prueba para Sistemas de Machine Learning

Los componentes no relacionados con IA de un MLS pueden probarse usando niveles de prueba convencionales. Además, los modelos ML y los MLS requieren pruebas adicionales para abordar los riesgos específicos asociados con ML.

Se utilizan dos niveles de prueba especializados para abordar los riesgos específicos de ML: - Pruebas de datos de entrada (véase Capítulo 5): Relacionadas con los datos de entrenamiento utilizados para entrenar un modelo ML y los datos de producción utilizados por un MLS para generar una predicción en el entorno operativo. - Pruebas de modelo ML (véase Capítulo 6): Relacionadas con las pruebas de los modelos ML, que son el resultado final del flujo de trabajo ML (véase 3.1.2).

Los riesgos asociados con ML no pueden abordarse todos usando estos dos niveles de prueba específicos de ML. Los siguientes niveles de prueba también son típicamente requeridos, dependiendo de los riesgos percibidos: - Pruebas de Componente: Aplicables a cualquier componente no relacionado con IA, como la interfaz de usuario, el pipeline de datos y los componentes de comunicación. - Pruebas de Integración de Componentes: Incluyen la verificación de que las entradas del pipeline de datos son recibidas según lo esperado por el modelo y que cualquier predicción generada por el modelo se intercambia con los componentes relevantes del sistema (por ejemplo, la interfaz de usuario) y se usa correctamente. Donde la IA se proporciona como servicio (véase 1.1.6), las pruebas de API del servicio proporcionado se realizan como parte de las pruebas de integración de componentes. - Pruebas de Sistema: Incluyen pruebas de confirmación de que el rendimiento funcional ML de las pruebas iniciales del modelo no se ve afectado adversamente cuando el modelo está integrado dentro de un sistema completo. Estas pruebas son especialmente importantes cuando el modelo ML ha sido deliberadamente modificado (por ejemplo, comprimiendo una DNN para reducir su tamaño). Las pruebas no funcionales también están cubiertas, por ejemplo, probar la eficiencia de rendimiento del tiempo requerido para entregar una

predicción de un sistema basado en IA completo. - Pruebas de Integración de Sistema: Se enfocan en verificar las interfaces y los intercambios de datos entre el sistema basado en IA y sistemas o servicios externos, usando un entorno representativo de las condiciones operativas. - Pruebas de Aceptación: Donde la IA se usa como servicio, las pruebas de aceptación pueden ser necesarias para determinar la idoneidad del servicio para el sistema previsto y si, por ejemplo, se han alcanzado los criterios de rendimiento funcional ML.

4.3.2 Pruebas Basadas en Riesgos de Sistemas de Machine Learning

Las pruebas basadas en riesgos deben aplicarse a todos los sistemas, independientemente de si contienen componentes de IA o no. La mayoría de los marcos regulatorios, ya sean publicados o en desarrollo, requieren un enfoque basado en riesgos para el desarrollo y gestión de sistemas basados en IA. Como los sistemas basados en IA presentan riesgos únicos, probarlos difiere de los sistemas no basados en IA.

No existe una forma estándar de categorizar los riesgos de ML; sin embargo, los riesgos específicos de ML están asociados tanto con el desarrollo del MLS (riesgos de proyecto) como con el MLS en sí (riesgos de producto). Una forma de categorizar estos riesgos es usar el flujo de trabajo ML y dividirlo en tres áreas principales: - Desarrollo - relacionado con el algoritmo ML, el desarrollo del modelo y el marco de desarrollo ML. Ejemplos de riesgos de proyecto incluyen selección subóptima de algoritmo, mala selección del enfoque de evaluación y vulnerabilidades de seguridad del framework. Véase el Capítulo 7 para más detalles. - Datos de entrada - relacionado con la provisión de datos de entrenamiento para apoyar ML y la provisión de datos de producción utilizados por el modelo en su entorno operativo. Ejemplos de riesgos de producto incluyen datos de entrenamiento sesgados, defectos en el pipeline de datos y datos de entrenamiento no representativos. Véase el Capítulo 5 para más detalles. - Modelo - relacionado con el modelo ML generado. Ejemplos de riesgos de producto incluyen el fracaso en alcanzar las medidas de rendimiento funcional ML requeridas, un modelo sobreajustado y

susceptibilidad a ejemplos adversariales. Véase el Capítulo 6 para más detalles.

Otra forma de categorizar los riesgos asociados con la IA es según las características de calidad definidas en ISO/IEC 25059.

Están disponibles varias formas de pruebas que abordan estos riesgos y están específicamente diseñadas para las pruebas de MLS. Por ejemplo, pruebas de pipeline de datos, pruebas adversariales y revisión de la idoneidad del algoritmo/modelo. Este syllabus cubre varias de estas en los capítulos 5, 6 y 7.

5 Pruebas de Datos de Entrada para Sistemas de Machine Learning - 180 minutos

Palabras clave: Data pipeline testing, data representativeness testing, dataset constraint testing, input data testing, label correctness testing, review, testing for bias

Palabras clave específicas de IA: Disparate impact analysis, multiple annotation

Objetivos de Aprendizaje para el Capítulo 5:

5.1 Pruebas de Datos de Entrada para Sistemas de Machine Learning - AI-5.1.1 (K2) Dar ejemplos de enfoques de prueba utilizados para la mitigación de riesgos de datos de entrada para un sistema de machine learning - AI-5.1.2 (K2) Explicar cómo probar el sesgo - AI-5.1.3 (K2) Resumir las diversas formas de pruebas de pipeline de datos - AI-5.1.4 (K2) Explicar cómo probar la representatividad de los datos - AI-5.1.5 (K3) Aplicar pruebas de restricciones de conjuntos de datos - AI-5.1.6 (K2) Explicar las pruebas de corrección de etiquetas - HO-5.1.7 (H2) Realizar pruebas de datos de entrada para conjuntos de datos ML

5.1 Pruebas de Datos de Entrada para Sistemas de Machine Learning

El objetivo de las pruebas de datos de entrada es confirmar que los datos utilizados por el MLS para entrenamiento, pruebas y predicción son de calidad suficiente (véase 3.2). Incluye revisiones, técnicas estadísticas (por ejemplo, pruebas de sesgo en los datos), EDA de los datos de entrenamiento, y pruebas estáticas y dinámicas del pipeline de datos.

5.1.1 Riesgos de Datos de Entrada y Mitigaciones

La siguiente tabla enumera ejemplos de riesgos de datos de entrada y las pruebas correspondientes que podrían usarse para la mitigación de riesgos:

Riesgos Potenciales	Posibles Mitigaciones de Riesgo
Defectos en los datos de entrenamiento que conducen a sesgo	Pruebas de sesgo – véase 5.1.2
Problemas dentro del algoritmo, modelo o marco de desarrollo ML que introducen injusticia sistémica	Pruebas de sesgo – véase 5.1.2
Datos de entrenamiento adquiridos de fuentes no confiables	Pruebas de procedencia de datos
Datos mal gestionados	Pruebas de procedencia de datos
Datos de entrenamiento envenenados	Pruebas A/B – véase 6.1.9; Pruebas de procedencia de datos; EDA – véase 3.2.1; Ataques como parte del red teaming – véase 4.2.2
Conjunto de datos internamente inconsistente	Pruebas de restricciones de conjuntos de datos – véase 5.1.5
Datos fuera de rango	

Riesgos Potenciales	Posibles Mitigaciones de Riesgo
	Pruebas de restricciones de conjuntos de datos – véase 5.1.5
Tipos de datos incorrectos	Pruebas de restricciones de conjuntos de datos – véase 5.1.5
Selección subóptima de características	Pruebas de características
Conjunto de datos desbalanceado debido a cobertura insuficiente de todas las clases objetivo	Pruebas de representatividad de datos – véase 5.1.4
Conjunto de datos sesgado por aumento de datos	Pruebas de representatividad de datos – véase 5.1.4
Datos faltantes	Pruebas de representatividad de datos – véase 5.1.4
Datos de entrenamiento enfocados en un subconjunto de todos los casos de uso	Pruebas de corrección de etiquetas – véase 5.1.6
Rango completo de valores no cubierto en el conjunto de datos	Pruebas de corrección de etiquetas – véase 5.1.6
Directrices de etiquetado deficientes	Pruebas de corrección de etiquetas – véase 5.1.6
Datos ambiguos	Pruebas de corrección de etiquetas – véase 5.1.6
Anotación deficiente que conduce a etiquetas inexactas o inconsistentes	Pruebas de corrección de etiquetas – véase 5.1.6
Diseño o integración deficiente que conduce a fallas en el pipeline de datos	Pruebas de pipeline de datos – véase 5.1.3
Defectos de calidad de datos que comprometen	Pruebas de pipeline de datos – véase 5.1.3

Riesgos Potenciales	Posibles Mitigaciones de Riesgo
las salidas del pipeline de datos	
Degradación del rendimiento en el uso operativo del pipeline de datos	Pruebas de pipeline de datos – véase 5.1.3
Brechas de seguridad o cambios no controlados que impactan el pipeline de datos	Pruebas de pipeline de datos – véase 5.1.3

5.1.2 Pruebas de Sesgo

El sesgo en un MLS se refiere a diferencias no aleatorias e injustas en el tratamiento basadas en atributos sensibles como género, edad o raza, lo cual es a menudo ilegal y hace que el sistema sea discriminatorio.

Las pruebas de sesgo en un MLS implican comprender sus fuentes potenciales, que incluyen principalmente: - Defectos en los datos de entrenamiento, como falta de representatividad, sesgos históricos o envenenamiento deliberado (sesgo de datos). - Defectos dentro del algoritmo, modelo o marco de desarrollo que introducen injusticia sistémica, como un algoritmo que usa un umbral de decisión para puntuaciones crediticias en un sistema de aprobación de préstamos (sesgo algorítmico).

Los enfoques de prueba para detectar sesgo incluyen los siguientes: - Revisiones del flujo de trabajo ML completo, y especialmente la preparación de datos, para identificar su potencial de introducir sesgo. - Revisiones de la documentación del conjunto de datos para identificar y mitigar posibles fuentes de injusticia examinando cómo se recopilaban y anotaban los datos, y qué poblaciones están representadas. - Análisis estático tanto de los programas de preparación de datos como del código de implementación del modelo para identificar anti-patrones o el manejo incorrecto de atributos sensibles. - EDA de los datos de entrenamiento usando métodos de visualización y

agrupamiento para revelar datos desbalanceados, distribuciones sesgadas o agrupaciones anómalas a través de diferentes atributos sensibles. - Pruebas dinámicas para ayudar a detectar sesgo en un modelo ML alimentando un conjunto de datos conocido, no sesgado y representativo a través del sistema y analizando sus predicciones en busca de diferencias estadísticamente significativas en los resultados entre varios grupos sensibles. Esto identifica sesgo en las salidas del modelo, independientemente de si el sesgo fue introducido durante el entrenamiento de datos o durante el desarrollo del modelo. - Pruebas de corrección de etiquetas (véase 5.1.6) para identificar etiquetado incorrecto que causa que se aprendan asociaciones incorrectas entre atributos y resultados para atributos sensibles específicos. - Análisis de impacto dispar: 1. Identificar atributos sensibles para el modelo. 2. Generar contrafactuales para los atributos sensibles (por ejemplo, escenarios de ejemplo donde el género se cambia de masculino a femenino en una solicitud de préstamo). 3. Generar la salida del modelo presentándole los contrafactuales. 4. Analizar los resultados de múltiples pruebas para lograr un resultado estadísticamente significativo, que determine si cambiar un atributo sensible causa que los resultados del modelo cambien, señalando la presencia de sesgo.

Nota: El análisis de impacto dispar también puede aplicarse a combinaciones de atributos sensibles, permitiendo identificar sesgo oculto asociado con combinaciones de atributos. Sin embargo, tenga cuidado de confirmar que los ejemplos contrafactuales no sean poco realistas, ya que el modelo puede entonces responder a ellos en lugar de a cualquier sesgo subyacente.

5.1.3 Pruebas de Pipeline de Datos

Las pruebas efectivas del pipeline de datos son esenciales no solo para confirmar la confiabilidad y el rendimiento de los sistemas basados en datos, sino también para mantener una alta calidad de datos a lo largo de todo el flujo de trabajo ML (véase 3.1.2).

Se emplea un enfoque por capas, comenzando con revisiones del diseño del pipeline de datos durante la fase de

diseño. Las pruebas de componentes incluyen pruebas de componentes de ingesta de datos, scripts de transformación e interfaces de sensores. Estas pruebas usan revisiones de código, análisis estático y pruebas específicas de hardware para verificar la captura confiable de datos. Las pruebas de componentes validan la lógica de transformación de datos, prueban la implementación de reglas de validación de datos, confirman el manejo robusto de errores y prueban vulnerabilidades que podrían explotarse para introducir malware o datos envenenados.

Las pruebas de integración de componentes verifican el flujo continuo de datos a través de interfaces internas y la interpretación correcta de los datos a medida que se mueven a través del pipeline. Detectan defectos que surgen de interfaces no coincidentes o suposiciones incorrectas entre componentes.

Las pruebas de sistema evalúan el pipeline completamente ensamblado, comenzando con pruebas de humo para confirmar la funcionalidad básica del pipeline. Las pruebas funcionales verifican el cumplimiento del pipeline con los requisitos especificados, incluyendo transformaciones de datos y enrutamiento de datos. Las pruebas no funcionales evalúan el rendimiento bajo carga, la escalabilidad para manejar volúmenes crecientes de datos y las medidas de seguridad para proteger la integridad de los datos. Las pruebas de inyección de fallas miden la robustez del pipeline simulando entradas de datos defectuosas, evaluando la capacidad del sistema para mantener la integridad de los datos cuando se enfrenta a datos inesperados o corruptos. Las pruebas back-to-back (véase 6.1.10) comparan el pipeline operativo contra el pipeline de entrenamiento para verificar la funcionalidad consistente.

Las pruebas de integración de sistema verifican la interacción correcta entre el pipeline de datos y sistemas o servicios externos, incluyendo fuentes de datos, plataformas de almacenamiento, herramientas de monitoreo y consumidores posteriores como los modelos ML.

Las pruebas en producción se realizan en el sistema operativo. Las pruebas back-to-back verifican que el

rendimiento es consistente o mejorado en comparación con versiones anteriores. Las pruebas A/B (véase 6.1.9) permiten comparaciones de nuevas iteraciones del pipeline contra líneas base, validando mejoras mientras se confirma que no hay degradación en el flujo de datos en vivo. Se pueden integrar herramientas para monitorear y observar continuamente el comportamiento del modelo, el rendimiento y los posibles defectos en tiempo real.

Las revisiones de gestión de configuración ayudan a verificar que se usen las versiones correctas del código del pipeline, las configuraciones y los conjuntos de datos a través del entrenamiento, las pruebas y la producción.

Finalmente, las estrategias de prueba deben alinearse con el propósito del pipeline. Los pipelines de entrenamiento, a menudo prototipos exploratorios, tienen prioridades diferentes a los pipelines operativos robustos. Las pruebas de pipelines de entrenamiento pueden enfocarse en la integridad de los datos, mientras que las pruebas de pipelines operativos priorizan la confiabilidad, el rendimiento y la mantenibilidad.

5.1.4 Pruebas de Representatividad de Datos

Las pruebas de representatividad de datos determinan qué tan estrechamente las características de los conjuntos de datos utilizados para entrenar, validar y probar modelos ML coinciden con los datos del mundo real que el modelo ML encontrará en operación. Estas pruebas abordan varias formas de falta de representación, incluyendo conjuntos de datos sesgados, datos faltantes, distribuciones desproporcionadas de características, cobertura inadecuada de escenarios operativos y representación desequilibrada de clases (véase 5.1.1).

Estas pruebas típicamente incluyen los siguientes pasos:

1. Definir la población objetivo:
2. Comprender los casos de uso previstos y el contexto operativo del MLS
3. Analizar las características de los usuarios finales y los entornos operativos

4. Identificar las distribuciones de datos operativos esperadas y los casos extremos críticos mediante:
 - consultar expertos del dominio para comprender los patrones de datos del mundo real
 - analizar datos de sistemas existentes o aplicaciones similares
 - usar conjuntos de datos de referencia de fuentes confiables (por ejemplo, NIST, bases de datos de la industria)
5. Aplicar muestreo estratificado a los datos de referencia representativos para crear una línea base que cubra todos los subgrupos relevantes en el dominio objetivo
6. Analizar las características de los datos:
7. Aplicar EDA (véase 3.2.1) a:
 - conjuntos de datos de entrenamiento/prueba que se evalúan para representatividad
 - conjunto de datos de referencia que representa los datos operativos esperados
 - visualizar distribuciones a través de histogramas, diagramas de dispersión y otras técnicas gráficas
8. Examinar las relaciones entre características, y las correlaciones en particular, para identificar patrones que deben preservarse
9. Identificar posibles anomalías, vacíos o concentraciones inusuales en los datos
10. Aplicar técnicas de evaluación estadística:
11. Usar pruebas estadísticas formales como Chi-cuadrado y Kolmogorov-Smirnov para comparar distribuciones [STATS]
12. Verificar desequilibrios en los datos, particularmente en problemas de clasificación
13. Verificar la cobertura adecuada tanto de escenarios típicos como de casos extremos/límite

Las pruebas de representatividad de datos deben realizarse antes del entrenamiento del modelo para evitar construir modelos sobre datos no representativos. Después del despliegue, las propiedades de los datos de entrada operativos deben monitorearse continuamente para detectar cambios que puedan indicar deriva de datos respecto a las distribuciones originales de datos de entrenamiento (véase 6.1.7 sobre pruebas de deriva).

5.1.5 Pruebas de Restricciones de Conjuntos de Datos

Las pruebas de restricciones de conjuntos de datos verifican si los datos en un conjunto de datos se adhieren a reglas o restricciones predefinidas. El objetivo es confirmar la integridad y consistencia de los datos utilizados en ML. Por ejemplo, podría realizarse una prueba de la consistencia de valores en un conjunto de datos.

Las restricciones sobre los datos se encuentran típicamente en esquemas de bases de datos. Un esquema de base de datos define la estructura, tipos y relaciones de los datos. De manera similar, para conjuntos de datos ML, se puede definir un conjunto de restricciones que actúan como un modelo lógico de los datos en el conjunto de datos, que debería satisfacerse si los datos son correctos. Hay varias formas de categorizar las restricciones de conjuntos de datos. Una restricción puede aplicarse a un único valor para un atributo, que se encuentra en una única instancia (restricción de valor único), por ejemplo: - Faltante - prueba valores faltantes y atributos faltantes. - Rango - prueba que un valor esté en un rango dado. - Tipo - prueba que un valor de atributo coincida con el tipo especificado (por ejemplo, que si un atributo se especifica como entero, el valor proporcionado no sea una cadena o un número real).

Alternativamente, una restricción puede aplicarse a través de múltiples valores, típicamente considerando los valores para un único atributo a través de varias instancias (restricción de múltiples valores), por ejemplo: - Suma - prueba que la suma de todos los valores sea igual, exceda o no exceda un valor especificado (por ejemplo, el valor total de todos los puntos otorgados para una carrera de Fórmula 1

no puede exceder 102, y debe exceder 50.5 puntos). - Conteo - prueba que el conteo de todos los valores no nulos para atributos o instancias sea igual, exceda o no exceda un valor especificado. - Duplicado - prueba valores de atributo o instancias idénticas o casi idénticas en un conjunto de datos e impone un límite sobre cuántos se permiten (a menudo cero). - Útil - prueba que un atributo contenga algunos valores repetidos, ya que si cada entrada es única (como un ID o una marca de tiempo), típicamente no proporciona patrones útiles para ser aprendidos por un modelo ML. - Atípico - identifica cualquier valor que pueda considerarse como atípico estadístico.

Una forma especial de restricción de múltiples valores puede comparar diferentes valores (restricción de comparación), por ejemplo: - Mayor Que - prueba que un valor para un atributo sea mayor que un valor para un segundo atributo (por ejemplo, el conteo de líneas de código para un programa excede su conteo de líneas de código con defectos). - Correlación - prueba que los valores para un atributo se correlacionen con los valores para un segundo atributo (por ejemplo, todos los estudiantes con un valor de atributo de notas al menos 1.33 desviaciones estándar por encima de la media también tienen un valor de 'A' para su atributo de calificación).

La prueba de datos contra las restricciones definidas puede realizarse manualmente. Sin embargo, la escala de la actividad y el tamaño típicamente grande del conjunto de datos requerirían que esto se automatice como parte del pipeline de datos. Cuando se implementa como parte del pipeline, la herramienta que implementa las pruebas de restricciones de conjuntos de datos puede proporcionar informes a los científicos de datos (para anomalías en datos de entrenamiento) o al personal de operaciones (para problemas de datos operativos).

5.1.6 Pruebas de Corrección de Etiquetas

La corrección de las etiquetas de datos es esencial en el aprendizaje supervisado. Las etiquetas inexactas o inconsistentes socavan directamente el rendimiento y la generalización de los modelos ML.

Los enfoques comunes para las pruebas de corrección de etiquetas de datos incluyen:

- **Revisión por Expertos:** Los expertos del dominio o anotadores capacitados revisan manualmente una muestra de datos etiquetados. Evalúan la precisión de las etiquetas usando su conocimiento del dominio y directrices.
- **Anotación Múltiple:** Los puntos de datos son etiquetados independientemente por múltiples anotadores y se comparan usando una forma de pruebas back-to-back (véase 6.1.10). Los desacuerdos destacan defectos que necesitan investigación y resolución. El acuerdo entre anotadores (IAA) puede medirse usando métricas como el Kappa de Cohen o porcentaje simple de acuerdo [STATS]. Puntuaciones bajas de IAA pueden indicar defectos en las directrices de etiquetado, datos ambiguos o anotación deficiente.
- **Priorización Basada en Riesgos para Revisión y Anotación:** Tanto las revisiones por expertos como las anotaciones múltiples pueden enfocarse usando un enfoque basado en riesgos para identificar muestras para revisión y anotaciones múltiples. La priorización puede basarse en la probabilidad de etiquetado incorrecto, como con puntos de datos ambiguos (por ejemplo, no está claro a qué categoría pertenecen o están cerca de un límite entre clases), y los datos que tienen más probabilidad de impactar el éxito o la seguridad de la aplicación.
- **Análisis de Distribución de Datos:** Cuando existen conjuntos de datos comparables, comparar la distribución de etiquetas del conjunto de datos bajo prueba con conjuntos de datos similares puede revelar anomalías y etiquetas potencialmente incorrectas.
- **Pruebas Automatizadas Basadas en Reglas:** Se pueden implementar pruebas automatizadas basadas en reglas o restricciones de etiquetas predefinidas para ciertas tareas. Por ejemplo, verificar que los cuadros delimitadores (rectángulos usados para localizar objetos en imágenes) no se superpongan o se extiendan más allá de los límites de la imagen.
- **Análisis de Pérdida del Modelo:** Los puntos de datos que exhiben alta pérdida durante el entrenamiento del modelo – lo que significa que las predicciones del modelo para estos puntos se desvían sustancialmente de sus etiquetas verdaderas – pueden indicar etiquetado incorrecto. La alta pérdida refleja un error significativo, indicando que el modelo tiene dificultades para aprender la etiqueta asignada y señalando un posible defecto.
- **Análisis de Puntuación de Confianza del**

Modelo: Los puntos de datos con baja confianza de predicción de un modelo entrenado pueden estar mal etiquetados, ser ambiguos o estar fuera de la distribución de datos de entrenamiento del modelo.

El uso de una combinación de enfoques es a menudo más efectivo. Por ejemplo, las revisiones por expertos son valiosas para establecer directrices de etiquetado claras desde el principio. Las puntuaciones de IAA de anotaciones múltiples pueden luego refinar el proceso de etiquetado. Posteriormente, los enfoques basados en modelos pueden identificar defectos potenciales de etiquetas adicionales a medida que el modelo continúa desarrollándose.

5.1.7 Ejercicio Práctico: Pruebas de Datos de Entrada

Para un conjunto de datos dado (por ejemplo, datos estructurados y tabulares), realizar pruebas de datos de entrada para cosas como datos faltantes, datos duplicados y valores atípicos.

6 Pruebas de Modelo para Sistemas de Machine Learning - 225 minutos

Palabras clave: A/B testing, adversarial testing, back-to-back testing, concept drift, data drift, drift testing, metamorphic testing, ML functional performance, ML model testing, review

Palabras clave específicas de IA: Overfitting, underfitting

Objetivos de Aprendizaje para el Capítulo 6:

6.1 Pruebas de Modelo para Sistemas de Machine Learning -
AI-6.1.1 (K2) Dar ejemplos de enfoques de prueba utilizados para la mitigación de riesgos de modelos ML - AI-6.1.2 (K2) Explicar el propósito y enfoque de la revisión de documentación de modelos ML - AI-6.1.3 (K2) Explicar cómo se llevan a cabo las pruebas de rendimiento funcional ML para sistemas de machine learning probabilísticos - AI-6.1.4

(K2) Resumir las pruebas adversariales de sistemas de machine learning - AI-6.1.5 (K3) Usar pruebas metamórficas para derivar casos de prueba para un escenario dado - HO-6.1.6 (H2) Aplicar pruebas metamórficas - AI-6.1.7 (K2) Explicar cómo se usan las pruebas de deriva en sistemas de machine learning operativos - AI-6.1.8 (K2) Explicar cómo se detectan el sobreajuste y el subajuste mediante pruebas - AI-6.1.9 (K2) Explicar cómo se usan las pruebas A/B en el contexto de sistemas de machine learning - AI-6.1.10 (K2) Explicar cómo se usan las pruebas back-to-back en el contexto de sistemas de machine learning

6.1 Pruebas de Modelo para Sistemas de Machine Learning

Las pruebas de modelo ML implican abordar riesgos específicos. Estos incluyen riesgos funcionales como sesgo, sobreajuste y vulnerabilidades adversariales, así como riesgos no funcionales como falta de robustez de IA y eficiencia de rendimiento, y riesgos de despliegue.

6.1.1 Riesgos de Modelos de Machine Learning y Mitigaciones

La siguiente tabla enumera ejemplos de riesgos de modelos ML y las pruebas correspondientes que podrían usarse para la mitigación de riesgos:

Riesgo Potencial	Posible Mitigación de Riesgo
Modelo ML sesgado o injusto	Pruebas de sesgo – véase 5.1.2
Modelo no ético	Pruebas éticas del sistema
Ejemplos adversariales	Pruebas adversariales – véase 6.1.4
Modelo sobreajustado	Pruebas de sobreajuste – véase 6.1.8
Modelo subajustado	Pruebas de subajuste – véase 6.1.8

Riesgo Potencial	Posible Mitigación de Riesgo
Deriva de datos inaceptable	Pruebas de deriva – véase 6.1.7
Deriva de concepto inaceptable	Pruebas de deriva – véase 6.1.7
El modelo causa efectos secundarios	Pruebas de efectos secundarios
El modelo exhibe hackeo de recompensa	Pruebas de hackeo de recompensa
Defecto en la API del modelo	Pruebas de API – véase 7.1.2
Fracaso en alcanzar las medidas de rendimiento requeridas del modelo ML (por ejemplo, falta de precisión, recall)	Pruebas de rendimiento funcional ML – véase 6.1.3
Incorrección funcional	Pruebas metamórficas – véase 6.1.5
Defectos no funcionales	Pruebas metamórficas – véase 6.1.5
Problema del oráculo de prueba	Pruebas metamórficas – véase 6.1.5; Pruebas back-to-back – véase 6.1.10; Pruebas A/B – véase 6.1.9
Requisitos del sistema deficientes	Revisión de requisitos; Red teaming – véase 4.2.2; Pruebas exploratorias
Falta de robustez de IA del modelo debido a entradas inesperadas	Pruebas adversariales – véase 6.1.4; Fuzz testing
Eficiencia de rendimiento inadecuada del modelo ML	Pruebas de rendimiento
Documentación deficiente del modelo (por ejemplo, función, precisión, interfaz)	Revisión de documentación del modelo – véase 6.1.2

Riesgo Potencial	Posible Mitigación de Riesgo
Actualizaciones del modelo introducen defectos	Pruebas back-to-back – véase 6.1.10
Actualizaciones del modelo disminuyen el rendimiento funcional del modelo ML	Pruebas A/B – véase 6.1.9
Despliegue del modelo actualizado causa fallo inmediato	Pruebas de humo
Despliegue del modelo actualizado causa regresión	Pruebas de regresión
Vulnerabilidades de seguridad	Red teaming – véase 4.2.2
Vulnerabilidades de protección	Red teaming – véase 4.2.2
Violaciones de privacidad	Red teaming – véase 4.2.2
Salidas dañinas o indeseables (por ejemplo, opiniones racistas, orientación peligrosa)	Red teaming – véase 4.2.2

6.1.2 Documentación y Revisión de Modelos de Machine Learning

La documentación exhaustiva para modelos ML no es solo una formalidad; es un recurso crítico. A diferencia de los sistemas donde el código fuente puede inspeccionarse directamente, los MLS presentan desafíos únicos debido a la comprensibilidad limitada del código generado por máquinas, la naturaleza inherente de caja negra de los modelos y su dependencia de los datos. Los modelos también se actualizan frecuentemente, por lo que la documentación se convierte en la herramienta principal para que desarrolladores, probadores y reguladores comprendan, evalúen y confíen en los sistemas basados en IA. La transparencia juega un papel central en permitir esta comprensión, permitiendo a las partes interesadas rastrear el comportamiento del modelo, la lógica de decisión y el linaje de datos a lo largo del ciclo de vida de la IA.

La documentación estandarizada mejora la comunicación, apoya la toma de decisiones informada y verifica la calidad y mantenibilidad de los modelos ML. Es cada vez más importante para el cumplimiento regulatorio, como lo evidencian marcos como el EU AI Act, que enfatiza las obligaciones de transparencia que requieren documentación clara de las decisiones del modelo, limitaciones y medidas de interpretabilidad. Para sistemas de alto riesgo, pasar una auditoría de documentación es a menudo un prerrequisito para el despliegue, mientras que para todos los sistemas, una revisión exhaustiva ayuda a verificar la calidad.

Existen varios marcos de documentación, incluyendo Model Cards, que proporcionan resúmenes concisos de los usos previstos de un modelo, resultados de evaluación y consideraciones éticas [MODEL_DOC], y Datasheets for Datasets, que ofrecen formatos estandarizados para describir conjuntos de datos, incluyendo su motivación, composición, proceso de recopilación y usos [DATA_DOC].

La siguiente lista describe los contenidos típicos esperados para una documentación exhaustiva de modelos ML, estructurada para que pueda usarse como una lista de verificación práctica tanto por desarrolladores como por probadores para ayudar a lograr completitud, claridad y testabilidad:

- General: Identificadores, descripción, desarrollador, versión, fecha, contacto, licencia, necesidades de hardware
- Diseño: Suposiciones, decisiones técnicas, algoritmo ML
- Uso: Propósito previsto, usos primarios/secundarios, usuarios, enfoque de autoaprendizaje, sesgo, ética, seguridad, transparencia, umbrales, plataforma, deriva de datos, deriva de concepto
- Conjuntos de datos: Características, fuente, recopilación, disponibilidad, preprocesamiento, uso, contenido, etiquetas, tamaño, privacidad, seguridad, sesgo/equidad, restricciones
- Pruebas: Detalles del conjunto de datos de prueba, independencia de las pruebas, resultados de pruebas, actividades de prueba (por ejemplo, funcionales, adversariales)
- Funcional: Medidas, conjunto de datos de validación, umbrales, rendimiento real
- No Funcional: Escalabilidad, confiabilidad, disponibilidad, eficiencia de rendimiento (por ejemplo, latencia, utilización de recursos), mantenibilidad, robustez de IA
- Operativo: Plan de

despliegue, entorno de despliegue, recursos computacionales, métricas/alertas de monitoreo, estrategia de reentrenamiento, plan de actualización/reversión del modelo, plan de obsolescencia, seguridad (riesgos adversariales), métodos de explicabilidad

Revisar la documentación contra tales listas de verificación es una actividad central de prueba, con el objetivo de: - Identificar información faltante, inexactitudes e inconsistencias. - Mejorar la claridad y legibilidad. - Mejorar la mantenibilidad identificando dónde la documentación necesita mejoras. - Proporcionar información suficiente para las actividades de prueba y despliegue. - Verificar que se han cumplido todos los requisitos regulatorios relevantes.

6.1.3 Pruebas de Rendimiento Funcional ML de Sistemas de Machine Learning Probabilísticos

Las pruebas de rendimiento funcional ML evalúan qué tan bien un modelo ML realiza sus funciones previstas midiendo métricas como accuracy, recall, precision y F1-score (véase 3.3) y comparando los resultados contra criterios de aceptación definidos.

Estas pruebas para MLS probabilísticos van más allá de un simple estado de aprobado/fallido para medir estadísticamente el rendimiento de un modelo contra sus criterios de aceptación [STATS]. Este enfoque es necesario para abordar el no determinismo inherente en los MLS evaluando el comportamiento a través de un conjunto de datos grande y representativo (véase 4.1.2).

Una precondition para estas pruebas es tener criterios de aceptación definidos en términos estadísticos. En lugar de un objetivo simple, un requisito podría especificar una métrica de rendimiento, un margen de error (MoE) y un nivel de confianza (CL). Tal requisito puede usarse para determinar el número mínimo de pruebas requeridas. A medida que aumenta el número de pruebas, hay dos opciones: - Fijar el CL y el MoE disminuirá. Esto significa que el resultado de la prueba se vuelve más preciso. - Fijar el MoE y el CL aumentará. Esto significa que el resultado de la prueba se vuelve más certero.

Por ejemplo, un criterio podría ser "98% de accuracy con un MoE de $\pm 4\%$ a un CL del 95%". Para confirmar que la accuracy medida tiene un MoE máximo de $\pm 4\%$ al CL del 95%, se requiere un tamaño de muestra de 601 casos de prueba. Esto se calcula usando la fórmula de tamaño de muestra para estimar una proporción poblacional. Este tamaño de muestra se basa en la suposición conservadora de que la accuracy verdadera podría estar entre 0% y 100%, lo que produce la mayor incertidumbre posible pero aún garantiza el MoE de $\pm 4\%$ bajo todas las condiciones. Para cumplir con la accuracy objetivo del 98%, al menos 589 de los 601 casos de prueba deben pasar. Si, después de ejecutar los 601 casos de prueba, la accuracy observada es del 98%, entonces el MoE medido al CL del 95% será más estrecho que $\pm 4\%$ (aproximadamente $\pm 1.1\%$), porque la varianza es menor a niveles altos de accuracy. El MoE se calcula usando la fórmula de margen de error para una proporción muestral. Esto significa que al probar, no siempre es necesario ejecutar los 601 casos de prueba. Las pruebas secuenciales proporcionan un marco estadístico formal para esta detención temprana. En lugar de ejecutar siempre la muestra fija completa, estos enfoques analizan los resultados a medida que se acumulan y se detienen tempranamente cuando hay evidencia suficiente que respalda el objetivo de accuracy (o lo rechaza). Si la accuracy observada permanece consistentemente alta (por ejemplo, no menos del 98%), entonces la incertidumbre estadística disminuye a medida que se ejecutan más pruebas. En esa situación, el MoE requerido de $\pm 4\%$ al CL del 95% se alcanza después de aproximadamente 170 casos de prueba, permitiendo que las pruebas concluyan antes.

NOTA: Las fórmulas y cálculos numéricos en este ejemplo (para tamaño de muestra, margen de error e intervalos de confianza) se proporcionan solo para ilustración y comprensión; no se requerirá que los candidatos deriven o calculen estos valores usando fórmulas estadísticas en el examen.

Para sistemas críticos de seguridad, podría usarse un requisito de confiabilidad más estricto, como "99% de confiabilidad con 95% de confianza", que requeriría 299

casos de prueba, todos los cuales deben pasar para cumplir con los criterios.

Para validar estos criterios, las pruebas requieren un conjunto de datos de prueba grande que sea completamente independiente de los conjuntos de datos de entrenamiento y validación. Este conjunto de datos de prueba debe ser una muestra representativa del dominio de entrada operativo (véase 5.1.4) para verificar que la evaluación refleje condiciones del mundo real. Los casos de prueba se ejecutan luego usando el modelo ML dentro de un marco de desarrollo ML capaz de análisis estadístico. Finalmente, los resultados agregados se interpretan y reportan con confianza estadística, no como una simple relación de aprobado/fallido. Un informe final de prueba declararía, por ejemplo, "El modelo alcanzó una accuracy del 94% $\pm$ 4% al CL del 95%." Esto permite a las partes interesadas comprender el rango de rendimiento operativo esperado y tomar una decisión informada sobre si el rendimiento funcional ML del modelo es aceptable para el despliegue.

6.1.4 Pruebas Adversariales de Sistemas de Machine Learning

Las pruebas adversariales implican proporcionar deliberadamente al modelo perturbaciones a los datos de entrada que a menudo son imperceptibles para los humanos. Estas entradas están diseñadas para causar que el modelo haga predicciones incorrectas, y, si tienen éxito, se llaman ejemplos adversariales. Así, las entradas de prueba para pruebas adversariales, y por lo tanto los posibles ejemplos adversariales, a menudo consisten en versiones ligeramente modificadas de entradas legítimas que causan que el modelo las clasifique incorrectamente.

La identificación de vulnerabilidades a través de pruebas adversariales permite a los desarrolladores incorporar salvaguardas y hacer el modelo más robusto contra ejemplos adversariales. Estos pueden ser ejemplos adversariales accidentales que encuentra, o pueden ser ataques adversariales que implican el uso malicioso de ejemplos adversariales. Generar entradas de prueba efectivas para pruebas adversariales es técnicamente complejo, y

mantenerse actualizado con las técnicas de ataque en evolución es un desafío continuo.

Las pruebas adversariales pueden realizarse usando pruebas de caja negra, enfocándose en el comportamiento de entrada y salida del modelo sin requerir conocimiento de su funcionamiento interno. Esto puede lograrse creando un modelo equivalente para el cual se conocen los internos, a partir del cual se pueden crear entradas de prueba adversariales a través del conocimiento de su funcionamiento interno. Luego, basándose en la suposición de que los modelos equivalentes comparten límites de clasificación (es decir, transferibilidad), las pruebas adversariales pueden aplicarse al modelo original. En contraste, un enfoque de fuerza bruta usando un gran número de pruebas puede usarse con la esperanza de que algunas pruebas aleatorias coincidan con un ejemplo adversarial.

Las pruebas adversariales también pueden realizarse usando pruebas de caja blanca. Comprender los internos de un modelo ML (arquitectura, parámetros, proceso de entrenamiento) típicamente facilita la creación de ejemplos adversariales. Las pruebas adversariales pueden hacerse manualmente creando ejemplos adversariales específicos o a través de algoritmos automatizados que generan un gran número de variaciones para encontrar entradas adversariales efectivas.

6.1.5 Pruebas Metamórficas

Las pruebas metamórficas (MT) son una técnica de prueba en la cual nuevos casos de prueba (de seguimiento) se derivan de un caso de prueba fuente previamente aprobado. Uno o más casos de prueba de seguimiento se generan cambiando (metamorfizando) el caso de prueba fuente usando una relación metamórfica (MR). La MR se basa en una propiedad de una función requerida del objeto de prueba, y describe cómo un cambio en las entradas de un caso de prueba se refleja en los resultados esperados del mismo caso de prueba.

MT puede usarse para la mayoría de objetos de prueba y puede aplicarse tanto a pruebas funcionales como no funcionales. Los probadores verifican objetivos de prueba como consistencia (las salidas se alinean entre entradas relacionadas), monotonía (las salidas cambian direccionalmente con la entrada) e invariancia (las salidas permanecen estables bajo perturbaciones). Es particularmente útil donde la generación de resultados esperados es problemática porque un oráculo de prueba asequible no está disponible. Este es el caso con algunos MLS que usan big data, o con sistemas donde los probadores no tienen claro cómo el modelo ML deriva sus predicciones, lo cual es frecuente. En el área de IA y ML, las pruebas metamórficas se han usado para probar el reconocimiento de imágenes, motores de búsqueda, optimización de rutas y reconocimiento de voz.

MT se selecciona típicamente sobre las pruebas tradicionales basadas en oráculo cuando: - no existen salidas esperadas confiables debido a la opacidad del modelo o la escala de datos; - el sistema es una caja negra; o - las propiedades relacionales (no valores absolutos) son suficientes para la confianza.

MT a menudo se basa en un caso de prueba fuente que pasó, y también puede ser útil cuando no es posible generar un resultado esperado. Por ejemplo, donde el programa implementa una función demasiado compleja para que un probador humano la replique y la use como oráculo de prueba, como con algunos MLS complejos. En esta situación, MT puede usarse para generar casos de prueba que, al ejecutarse, crearán un conjunto de salidas donde las relaciones entre las salidas (en lugar de sus valores reales) se verifican para validez. Con esta forma de MT, si las relaciones entre las salidas de prueba se mantienen verdaderas, proporciona mayor confianza en el programa. Por ejemplo, un MLS de evaluación de riesgos que predice una edad de muerte, donde aumentar el número de cigarrillos fumados debería disminuir la predicción (monotonía).

Los probadores derivan MRs del conocimiento del dominio, requisitos o propiedades del dominio (por ejemplo, leyes de

la física). Las MRs pueden validarse mediante revisión de expertos, ejecutándolas en modelos de referencia y verificando la cobertura de casos extremos.

MRs incorrectas (por ejemplo, pasar por alto interacciones complejas entre variables) o conjuntos incompletos de MRs pueden llevar a falsa confianza. MT detecta fallas relacionales pero no todos los errores absolutos, por lo que debe usarse en combinación con otras técnicas de prueba.

6.1.6 Ejercicio Práctico: Aplicar Pruebas Metamórficas

En este ejercicio, los estudiantes adquirirán experiencia práctica en lo siguiente: - Derivar varias MRs para un MLS dado. Estas MRs deben incluir algunas en las que los resultados esperados de los casos de prueba fuente y de seguimiento sean iguales, y algunas en las que sean diferentes. - Generar casos de prueba fuente para el MLS. No es necesario garantizar que pasen, pero se debe recordar a los estudiantes las limitaciones de MT cuando no se dispone de casos de prueba fuente que hayan pasado. - Usar las MRs derivadas y los casos de prueba fuente generados para derivar casos de prueba de seguimiento. - Ejecutar los casos de prueba de seguimiento.

6.1.7 Pruebas de Deriva

Las pruebas de deriva pueden usarse para identificar dos formas de deriva en MLS operativos: - Deriva de datos - ocurre cuando las propiedades estadísticas de los datos de entrada operativos cambian con el tiempo. Por ejemplo, los datos de entrada son ahora significativamente diferentes de los datos con los que se entrenó el modelo, debido a factores como cambios en el comportamiento del usuario o estacionalidad. Por ejemplo, un filtro de spam encuentra nuevos tipos de ataques de phishing que no existían durante su entrenamiento. - Deriva de concepto - ocurre cuando la relación entre los datos de entrada y la salida correcta cambia con el tiempo. Esto significa que los patrones o reglas originalmente aprendidos por el modelo ya no reflejan la realidad actual. Por ejemplo, debido a nuevas regulaciones financieras, un tipo de transacción previamente considerado

'bajo riesgo' ahora podría clasificarse como 'alto riesgo'. El significado de los datos ha cambiado, causando que los límites de decisión aprendidos por el modelo se vuelvan obsoletos, llevando a una disminución en su precisión predictiva.

Las pruebas de deriva dinámicas dependen de la disponibilidad de retroalimentación de los usuarios, que proporciona la verdad fundamental actual. Esta verdad fundamental actual se compara con la salida del modelo, y la diferencia entre ambas se determina y se compara contra un valor umbral. La retroalimentación del usuario puede ser directa o indirecta. Para un sistema de recomendación de películas, un ejemplo de retroalimentación directa es cuando el usuario califica una recomendación. Un ejemplo de retroalimentación indirecta para el mismo sistema sería extraída de los datos sobre las películas que el usuario ha visto.

Las pruebas de deriva estáticas no dependen de la verdad fundamental actual sino que comparan las propiedades estadísticas de las distribuciones de datos de entrada y salida predicha usando una prueba como Kolmogorov-Smirnov [STATS]. Una diferencia significativa en cualquiera de estas distribuciones es un indicador de que ha ocurrido deriva.

6.1.8 Pruebas de Sobreajuste y Subajuste

El sobreajuste y el subajuste son dos de los tres posibles resultados encontrados en modelos ML, siendo el tercero un modelo con "ajuste correcto". Las pruebas de sobreajuste y subajuste deben ocurrir durante el entrenamiento, la evaluación y el ajuste.

El sobreajuste ocurre cuando un modelo aprende los datos de entrenamiento demasiado bien, al punto de que captura ruido en los datos en lugar del patrón subyacente. Esto resulta en una pobre generalización a datos nuevos y no vistos.

Para probar el sobreajuste, el rendimiento funcional ML del modelo se evalúa en un conjunto de datos de prueba separado que no se usó durante el entrenamiento. Este

conjunto de datos de prueba debe incluir algunos ejemplos menos comunes que probablemente no se usaron durante el entrenamiento. El modelo podría estar sobreajustado si se desempeña significativamente peor en el conjunto de datos de prueba que en el conjunto de datos de validación.

El subajuste ocurre cuando un modelo es demasiado simple para capturar la estructura subyacente de los datos o cuando los datos de entrenamiento no contienen características que reflejen una relación importante entre entradas y salidas, resultando en un rendimiento funcional ML deficiente tanto en los conjuntos de datos de entrenamiento como de validación.

Durante las pruebas, el subajuste puede detectarse evaluando las métricas de rendimiento funcional ML del modelo como accuracy, precision, recall o F1 score. Si estas métricas son consistentemente bajas tanto en los conjuntos de entrenamiento como de validación, sugiere que el modelo está subajustado.

La inspección visual de las curvas de aprendizaje del modelo también puede ayudar a detectar el subajuste. Si los errores de entrenamiento y validación permanecen altos y relativamente cercanos, sin mejora significativa a medida que el entrenamiento progresa, indica subajuste.

En resumen, detectar el sobreajuste y subajuste durante las pruebas implica evaluar el rendimiento funcional ML del modelo en datos de validación, analizar métricas de rendimiento funcional ML y examinar curvas de aprendizaje.

6.1.9 Pruebas A/B

Las pruebas A/B son un enfoque donde la respuesta de dos variantes del programa (A y B) a las mismas entradas se compara con el propósito de determinar cuál de las dos variantes es mejor. Es un enfoque de pruebas estadísticas que típicamente requiere comparar resultados de pruebas de múltiples ejecuciones para determinar diferencias entre los programas.

Un ejemplo simple de este enfoque es donde dos ofertas promocionales se envían por correo electrónico a una lista de

marketing dividida en dos conjuntos. La mitad de la lista recibe la oferta A, y la otra mitad recibe la oferta B; el éxito de cada oferta ayuda a decidir cuál usar en el futuro. Muchas empresas de comercio electrónico y basadas en web usan pruebas A/B en producción, desviando diferentes consumidores a diferentes funcionalidades, para ayudar a identificar las preferencias de los consumidores.

Las pruebas A/B son un enfoque para abordar el problema del oráculo de prueba, típicamente usando el sistema existente como un oráculo de prueba parcial. Las pruebas A/B no generan casos de prueba y no proporcionan orientación sobre cómo deben diseñarse las pruebas, aunque las entradas operativas a menudo se incorporan en las pruebas.

Las pruebas A/B pueden usarse para probar actualizaciones a un sistema basado en IA, siempre que haya criterios de aceptación acordados, como métricas de rendimiento funcional ML, como se describe en 3.3. Cada vez que el sistema se actualiza, las pruebas A/B se usan para determinar que la variante actualizada se desempeña tan bien como, o mejor que, la variante anterior.

Tal enfoque de prueba puede usarse para un clasificador simple pero también puede usarse para probar sistemas mucho más complejos. Por ejemplo, una actualización para mejorar la efectividad de un sistema de enrutamiento de transporte de ciudad inteligente puede probarse usando pruebas A/B. Por ejemplo, comparando los tiempos promedio de desplazamiento para dos variantes del sistema en semanas consecutivas.

Las pruebas A/B también pueden usarse para probar sistemas de autoaprendizaje. Cuando el sistema hace un cambio, se ejecutan pruebas automatizadas, y las características del sistema resultante se comparan con las anteriores al cambio. Si el sistema mejora, el cambio se acepta; de lo contrario, el sistema revierte a su estado anterior.

Las técnicas estadísticas más populares para pruebas A/B son la prueba t, la prueba z, la prueba chi-cuadrado y la prueba U de Mann-Whitney [STATS].

6.1.10 Pruebas Back-to-Back

Las pruebas back-to-back ofrecen una solución práctica al problema del oráculo de prueba.

Las pruebas back-to-back implican usar una versión alternativa del sistema como punto de referencia (un pseudo-oráculo) y comparar sus salidas con las del sistema que se está probando cuando se presentan las mismas entradas. Este pseudo-oráculo podría ser un sistema existente, o uno desarrollado específicamente para pruebas, pero esto tiene un costo.

Idealmente, el pseudo-oráculo y el sistema bajo prueba no deberían compartir componentes de software comunes. De lo contrario, ambos sistemas podrían contener el mismo defecto, causando que sus salidas coincidan incluso cuando ambos están equivocados. Esto puede ser particularmente problemático dado el uso generalizado de componentes de IA reutilizables y de código abierto en el desarrollo de MLS. Por esta razón, el pseudo-oráculo es a menudo desarrollado por un equipo diferente e idealmente independiente, quizás usando diferentes marcos de desarrollo ML, algoritmos o configuraciones del modelo. A veces, el software convencional puede servir como pseudo-oráculo si resuelve el mismo problema.

Al realizar pruebas back-to-back funcionales, el pseudo-oráculo solo necesita coincidir en el comportamiento funcional. No necesita cumplir los mismos requisitos no funcionales que el sistema que se está probando, lo que potencialmente lo hace menos costoso de construir.

Este enfoque de prueba requiere solo generar entradas de prueba, no resultados esperados, ya que el pseudo-oráculo proporciona el punto de comparación. Estas entradas pueden provenir de casos de prueba existentes, como conjuntos de pruebas de regresión, o pueden generarse automáticamente a partir de datos de entrenamiento, permitiendo ejecutar un gran número de pruebas si se soporta la ejecución automatizada de pruebas.

Las pruebas back-to-back proporcionan un valor significativo al migrar un MLS a un nuevo entorno, como pasar de

desarrollo a producción, y al comparar resultados de pruebas entre entornos. Además, este enfoque de prueba puede revelar defectos sutiles en el comportamiento del modelo que podrían no ser aparentes a través de otros enfoques de prueba, especialmente al comparar respuestas a través de una amplia gama de casos extremos o entradas inusuales.

Una diferencia significativa entre las pruebas A/B (véase 6.1.9) y las pruebas back-to-back es el uso de las pruebas A/B para comparar dos variantes del mismo MLS usando métricas de rendimiento funcional ML y técnicas estadísticas, versus el uso de las pruebas back-to-back para detectar defectos.

7 Pruebas de Desarrollo de Machine Learning - 30 minutos

Palabras clave: ML development testing, ML functional performance, shadow testing

Palabras clave específicas de IA: Ninguna

Objetivos de Aprendizaje para el Capítulo 7:

7.1 Pruebas de Desarrollo de Machine Learning - AI-7.1.1 (K2)
Dar ejemplos de enfoques de prueba utilizados para la mitigación de riesgos del desarrollo ML - AI-7.1.2 (K2)
Explicar las diversas formas de pruebas de despliegue de sistemas ML

7.1 Pruebas de Desarrollo de Machine Learning

Este capítulo se enfoca específicamente en abordar los riesgos introducidos por las herramientas de desarrollo ML, las opciones de configuración y los mecanismos de despliegue, en lugar del modelo ML en sí. Cubre enfoques de prueba y tipos de prueba, como pruebas de API, pruebas de rendimiento funcional ML, pruebas A/B, pruebas back-to-

back y revisiones, para verificar la robustez de IA y la eficiencia.

7.1.1 Riesgos de Desarrollo de Machine Learning y Mitigaciones

La siguiente tabla enumera ejemplos de riesgos de desarrollo ML y las pruebas correspondientes que podrían usarse para la mitigación de riesgos:

Riesgo Potencial	Posible Mitigación de Riesgo
Uso incorrecto o no intencionado de APIs de bibliotecas o frameworks (por ejemplo, TensorFlow, PyTorch)	Pruebas de API – véase 7.1.2
Selección subóptima de framework	Revisión de idoneidad del framework
Problemas dentro del algoritmo, modelo o marco de desarrollo que introducen injusticia sistémica	Pruebas de sesgo – véase 5.1.2
Instalación o compilación defectuosa del framework	Pruebas de humo
Implementación defectuosa de la evaluación por el framework	Revisiones del código de evaluación del framework; Verificación cruzada de resultados de evaluación del framework (por ejemplo, contra benchmarks manuales)
Eficiencia de rendimiento deficiente (por ejemplo, el framework es lento para responder)	Pruebas de rendimiento
Usabilidad deficiente del framework	Pruebas de usabilidad
Defecto en una biblioteca usada por el framework	Pruebas de rendimiento funcional ML – véase 6.1.3;

Riesgo Potencial	Posible Mitigación de Riesgo
(por ejemplo, defecto en PyTorch)	Pruebas back-to-back – véase 6.1.10
Implementación defectuosa del algoritmo	Pruebas de rendimiento funcional ML – véase 6.1.3; Pruebas back-to-back – véase 6.1.10
Vulnerabilidades de seguridad en el framework	Pruebas de seguridad
Documentación deficiente para el usuario del framework	Revisión de documentación del framework
Selección subóptima de algoritmo	Revisión de idoneidad del algoritmo; Pruebas A/B – véase 6.1.9
Selección subóptima de hiperparámetros (por ejemplo, estructura de red, tasa de aprendizaje)	Pruebas de rendimiento funcional ML – véase 6.1.3; Pruebas A/B – véase 6.1.9
Asignación defectuosa de datos a conjuntos de datos de entrenamiento, validación y prueba	Revisión de asignación de datos
Mala selección del enfoque de evaluación (por ejemplo, validación cruzada k-fold)	Pruebas de rendimiento funcional ML – véase 6.1.3
Interpretación incorrecta de resultados de prueba debido a la naturaleza estocástica del proceso de aprendizaje	Pruebas de rendimiento funcional ML – véase 6.1.3
Defecto de despliegue (por ejemplo, al generar una versión modificada para una plataforma objetivo)	Pruebas de humo; Pruebas de rendimiento funcional ML – véase 6.1.3; Pruebas A/B – véase 6.1.9
Modelo desplegado es incompatible con el entorno operativo	Pruebas de humo; Pruebas de despliegue de MLS – véase 7.1.2

Riesgo Potencial	Posible Mitigación de Riesgo
Modelo desplegado no es una mejora sobre el modelo actual	Shadow testing – véase 7.1.2

7.1.2 Pruebas de Despliegue de Sistemas de Machine Learning

Desplegar MLS implica varias actividades clave de prueba enfocadas en verificar que el sistema basado en IA funciona correcta y confiablemente en su entorno objetivo (por ejemplo, nube, dispositivo edge, móvil). Cada uno de los siguientes tipos de prueba aborda riesgos específicos durante el despliegue:

- Pruebas de instalabilidad - verifican que el MLS puede instalarse, configurarse y posteriormente desinstalarse exitosamente en todos los entornos soportados. Esto incluye probar las dependencias del sistema (por ejemplo, drivers de GPU), compatibilidad con frameworks y la ejecución exitosa de scripts de instalación.
- Pruebas de reversión (rollback) - verifican la capacidad del sistema para revertir exitosamente a un estado previamente estable y operativo después de un despliegue degradado o fallido. Esto puede cubrir solo el modelo o cubrir el sistema completo (por ejemplo, incluyendo el pipeline de datos). Nótese que las pruebas de reversión deben realizarse antes del despliegue para confirmar la preparación para la reversión.
- Pruebas canary - validan nuevos despliegues liberando un modelo actualizado a un pequeño subconjunto del tráfico de producción (por ejemplo, 5% de los usuarios). Las métricas en tiempo real, como latencia, accuracy y tasas de error, se monitorean para detectar regresiones antes de un despliegue completo.
- Shadow testing - ejecuta un nuevo modelo en paralelo con el modelo de producción actual en tiempo real, enrutando las mismas solicitudes a

ambos sistemas sin afectar las respuestas en vivo. Permite comparar modelos nuevos y antiguos usando datos en vivo en un entorno controlado y de bajo riesgo, y puede descubrir defectos, como regresiones de rendimiento y deriva de datos, antes del despliegue completo.

- Pruebas de conversión de modelo - verifican que un modelo ML retiene accuracy predictiva aceptable, comportamiento consistente y eficiencia operativa (por ejemplo, velocidad de inferencia, uso de memoria) después de ser convertido de su formato de entrenamiento original a un formato de despliegue adecuado para el entorno de producción objetivo.
- Pruebas entre dispositivos (cross-device) - verifican que el MLS se desempeña correctamente en su rango previsto de objetivos de despliegue, desde dispositivos móviles y dispositivos edge hasta servidores en la nube.
- Pruebas de API - verifican que el MLS expone interfaces bien definidas y conformes a estándares. Valida el manejo correcto de entradas y salidas, mensajes de error y flujos de trabajo de integración con sistemas externos, como feeds de datos, clientes y el pipeline.

8 Lista de Abreviaturas

Abreviatura	Descripción
AI	inteligencia artificial
AlaaS	AI como servicio
API	interfaz de programación de aplicaciones
CL	nivel de confianza
CNN	red neuronal convolucional
CPU	unidad central de procesamiento

Abreviatura	Descripción
DL	aprendizaje profundo
DNN	red neuronal profunda
EDA	análisis exploratorio de datos
FN	falso negativo
FP	falso positivo
GAN	red generativa adversarial
GenAI	IA generativa
GPU	unidad de procesamiento gráfico
HO	objetivo práctico
IAA	acuerdo entre anotadores
kMNC	cobertura neuronal de k-multisección
LIME	explicaciones locales interpretables agnósticas al modelo
LLM	modelo(s) de lenguaje grande
LO	objetivo de aprendizaje
ML	machine learning
MLS	sistema(s) de machine learning
MoE	margen de error
NBC	cobertura de frontera neuronal
NLP	procesamiento de lenguaje natural
RAG	generación aumentada por recuperación
RNN	red neuronal recurrente
RT	red teaming
SVM	máquina de vectores de soporte
TN	verdadero negativo
TP	verdadero positivo

9 Términos Específicos de IA

Término	Definición
accuracy	La métrica de rendimiento funcional ML utilizada para evaluar un clasificador, que mide la proporción de predicciones que fueron correctas. (Según ISO/IEC TR 29119-11)
activation function	La fórmula asociada con una neurona en una red neuronal que determina la salida de la neurona a partir de las entradas a la neurona.
activation value	La salida de una función de activación de una neurona en una red neuronal.
adaptive AI-based system	Un sistema basado en IA que ajusta su comportamiento en respuesta a cambios en su entorno operativo.
adversarial attack	El uso deliberado de ejemplos adversariales para causar que un modelo ML falle.
AI as a Service	Un modelo de licenciamiento y entrega de software en el cual los servicios de IA y desarrollo de IA están alojados centralmente.
AI component	Un componente que proporciona funcionalidad de IA.
AI model	Un programa informático que implementa IA.
AI-based system	Un sistema que incorpora uno o más componentes de IA.
algorithmic bias	Un tipo de sesgo causado por el algoritmo ML.
annotation	La actividad de identificar objetos en imágenes con cuadros delimitadores para proporcionar datos etiquetados para clasificación.

Término	Definición
artificial intelligence	La capacidad de un sistema diseñado para adquirir, procesar, crear y aplicar conocimiento y habilidades. (ISO/IEC TR 29119-11)
association	Una técnica de aprendizaje ML no supervisado que identifica relaciones y dependencias entre muestras.
augmentation	La actividad de crear nuevos puntos de datos basados en un conjunto de datos existente.
Bayesian model	Un modelo estadístico que usa probabilidad para representar la incertidumbre tanto de las entradas como de las salidas del modelo.
bias	La diferencia sistemática en el tratamiento de ciertos objetos, personas o grupos en comparación con otros. (Según ISO/IEC TR 24027)
big data	Conjuntos de datos extensos cuyas características en términos de volumen, variedad, velocidad y/o variabilidad requieren tecnologías y técnicas especializadas para procesar.
bootstrap technique	Una técnica de remuestreo que extrae repetidamente muestras con reemplazo de un conjunto de datos de entrenamiento para estimar los criterios de rendimiento funcional ML de un modelo ML.
chatbot	Una aplicación utilizada para mantener una conversación vía texto o texto a voz.
classification	Una función ML que predice la clase de salida para una entrada dada. (Según ISO/IEC TR 29119-11)
classifier	Un modelo ML utilizado para clasificación. Sinónimo: modelo de clasificación
clustering	Una función ML que agrupa puntos de datos similares juntos.

Término	Definición
clustering algorithm	Un tipo de algoritmo ML utilizado para agrupar objetos similares en clusters.
concept drift	Un cambio en el rendimiento funcional percibido de un modelo ML con el tiempo causado por cambios en las expectativas del usuario, comportamiento y el entorno operativo.
confusion matrix	Una técnica para resumir el rendimiento funcional ML de un algoritmo de clasificación.
convolutional neural network	Un tipo de modelo de aprendizaje profundo diseñado para procesar datos en forma de cuadrícula como imágenes, permitiéndole reconocer patrones espaciales y características a través de operaciones en capas.
cross-prompt injection attack	Un ataque en el cual instrucciones maliciosas en un prompt o segmento de contexto interrumpen el comportamiento de un modelo de IA en un prompt, turno o segmento de contexto diferente.
data acquisition	La actividad de adquirir datos relevantes para el problema de negocio a resolver por un modelo ML.
data bias	Un error sistemático causado por datos de entrenamiento inexactos, incompletos o no representativos que conduce a resultados injustos en modelos ML.
data drift	Un cambio en la distribución de los datos de entrada con el tiempo, que puede impactar negativamente el rendimiento funcional de un modelo ML operativo.
data pipeline	La implementación de actividades de preparación de datos para proporcionar datos de entrada para apoyar el entrenamiento por un algoritmo ML o la predicción por un modelo ML.
data point	

Término	Definición
	Un conjunto de una o más mediciones que comprenden una única observación utilizada como parte de un conjunto de datos.
data preparation	Las actividades de adquisición de datos, preprocesamiento de datos e ingeniería de características en el flujo de trabajo ML.
data preprocessing	Las actividades de limpieza de datos, transformación de datos, aumento de datos y muestreo de datos en el flujo de trabajo ML.
dataset	Una colección de datos utilizada para entrenamiento, evaluación, pruebas y predicción en ML.
decision tree	Un modelo ML en forma de árbol cuyos nodos representan decisiones y cuyas ramas representan posibles resultados.
deep learning	ML que usa redes neuronales profundas para aprender automáticamente características y representaciones complejas de grandes conjuntos de datos.
deep neural network	Una red neuronal compuesta por varias capas de neuronas. Sinónimo: perceptrón multicapa
deepfake	Medios sintéticos, como video, audio o imágenes, creados o editados usando IA para imitar o suplantar de manera convincente personas o eventos reales.
defect prediction	Una técnica para predecir las áreas dentro del objeto de prueba en las cuales ocurrirán defectos o la cantidad de defectos presentes.
deterministic	Producir el mismo conjunto de salidas y estado final a partir de un conjunto dado de entradas y estado inicial.
disparate impact analysis	Una técnica para detectar sesgo comparando decisiones sobre escenarios originales con sus versiones

Término	Definición
	contrafactuales, donde los atributos sensibles se intercambian.
edge AI	El despliegue de modelos de IA en dispositivos edge locales, permitiendo procesamiento en tiempo real cerca de la fuente de datos sin depender de sistemas basados en la nube.
edge computing	La parte de una arquitectura distribuida en la cual el procesamiento de información se realiza cerca de donde esa información se utiliza.
epoch	Una iteración del entrenamiento ML sobre todo el conjunto de datos de entrenamiento.
expert system	Un sistema basado en IA para resolver problemas en un dominio o área de aplicación particular extrayendo inferencias de una base de conocimiento desarrollada a partir de la experiencia humana.
explainable AI	El campo de estudio relacionado con la comprensión de los factores que influyen en las salidas de sistemas de IA.
exploratory data analysis	Un proceso interactivo, visual e impulsado por hipótesis para resumir, explorar y comprender las principales características y patrones de los datos.
F1-Score	Una métrica de rendimiento funcional ML utilizada para evaluar un clasificador que proporciona un equilibrio entre recall y precision.
false negative	Una predicción del modelo ML en la cual el modelo predice erróneamente la clase negativa.
false positive	Una predicción del modelo ML en la cual el modelo predice erróneamente la clase positiva.
feature	

Término	Definición
	Un atributo individual medible de los datos de entrada utilizado para entrenamiento por un algoritmo ML y para predicción por un modelo ML.
feature engineering	La actividad en la cual aquellos atributos en los datos crudos que mejor representan las relaciones subyacentes que deben aparecer en el modelo ML se identifican para uso en los datos de entrenamiento. (ISO/IEC TR 29119-11)
feature testing	Un tipo de prueba para determinar si un conjunto de datos de entrenamiento de un modelo de IA contiene un conjunto apropiado de características.
foundation model	Un modelo ML a gran escala entrenado con grandes conjuntos de datos usando aprendizaje auto-supervisado, diseñado como una base versátil que puede ajustarse finamente o adaptarse a una amplia gama de tareas en diferentes dominios.
frontier AI	Un sistema basado en IA de propósito general que excede las capacidades de los sistemas de IA más avanzados de hoy.
fuzzy logic	Un tipo de lógica basada en el concepto de verdad parcial representada por factores de certeza entre 0 y 1.
general AI	Un tipo de IA que puede igualar las capacidades cognitivas humanas en la mayoría de las tareas intelectuales.
generative AI	Un tipo de IA que crea nuevo contenido aprendiendo patrones de datos existentes.
graphics processing unit	Un circuito integrado de aplicación específica diseñado para manipular y alterar la memoria para acelerar la creación de imágenes en un buffer de cuadro destinado a la salida hacia un dispositivo de visualización.

Término	Definición
ground truth	La información proporcionada por observación directa y medición que se sabe que es real o verdadera.
hyperparameter	Un parámetro utilizado ya sea para controlar el entrenamiento de un modelo ML o para establecer la configuración de un modelo ML.
hyperparameter tuning	La actividad de determinar los hiperparámetros óptimos basados en objetivos particulares.
intelligent agent	Un programa autónomo que dirige su actividad hacia el logro de objetivos usando observaciones y acciones.
inter-annotator agreement	El grado de consenso o similitud entre las anotaciones realizadas por diferentes anotadores sobre los mismos datos. (ISO/ IEC TS 12791)
large language model	Un sistema de IA generativa de texto a texto entrenado con colecciones muy grandes de datos de lenguaje.
linear regression	Una técnica estadística que modela la relación entre variables ajustando una ecuación lineal a los datos observados cuando la variable objetivo es numérica.
locked AI-based system	Un sistema basado en IA determinista con un modelo fijo e inmutable que no cambia su comportamiento una vez desplegado.
ML algorithm	Un algoritmo utilizado para crear un modelo ML a partir de un conjunto de datos de entrenamiento.
ML development framework	Una plataforma de software que proporciona herramientas y bibliotecas para construir, entrenar y desplegar modelos ML.
ML function	Funcionalidad implementada por un modelo ML, como clasificación, regresión ML o clustering.

Término	Definición
ML regression	Un tipo de función ML que resulta en un valor de salida numérico o continuo para una entrada dada. (Según ISO/IEC TR 29119-11)
MLS	Un sistema que integra uno o más modelos ML.
ML workflow	El conjunto de actividades utilizadas para desarrollar, desplegar y operar un modelo ML.
model confidence score analysis	Una técnica que identifica puntos de datos con puntuaciones de confianza bajas del entrenamiento, que son indicadores de puntos de datos mal etiquetados.
model loss analysis	Una técnica que identifica puntos de datos con valores de pérdida altos durante el entrenamiento, que son indicadores de puntos de datos mal etiquetados.
multimodal model	Un modelo ML diseñado para manejar múltiples tipos de modalidades de datos, como texto, imágenes, audio y video.
multiple annotation	Un enfoque en el cual puntos de datos etiquetados de múltiples anotadores se comparan.
narrow AI	IA enfocada en una única tarea bien definida para abordar un problema específico. Sinónimo: IA débil (ISO/IEC TR 29119-11)
natural language processing	Un campo de la computación que proporciona la capacidad de leer, comprender y derivar significado de lenguajes naturales.
neural network	Una red de elementos de procesamiento primitivos conectados por enlaces ponderados con pesos ajustables, en la cual cada elemento produce un valor aplicando una función no lineal a sus valores de entrada, y lo transmite a otros elementos o lo presenta como valor de

Término	Definición
	salida. Sinónimo: red neuronal artificial (ISO/IEC 2382)
neuromorphic processor	Un circuito integrado diseñado para imitar las neuronas biológicas del cerebro humano.
neuron	Un nodo en una red neuronal, usualmente recibiendo múltiples valores de entrada y generando un valor de activación.
noise	Una distorsión o corrupción en los datos.
non-determinism	Una propiedad de un sistema o proceso en el cual un resultado no está determinado únicamente por sus condiciones iniciales.
outlier	Una observación que se encuentra fuera del patrón general de la distribución de datos.
overfitting	La generación de un modelo ML que corresponde demasiado estrechamente al conjunto de datos de entrenamiento, resultando en un modelo que encuentra difícil generalizar a nuevos datos. (Según ISO/IEC TR 29119-11)
perceptron	Una red neuronal con solo una capa y una neurona.
precision	Una métrica de rendimiento funcional ML utilizada para evaluar un clasificador, que mide la proporción de positivos predichos que fueron correctos. (Según ISO/IEC TR 29119-11)
pretrained model	Un modelo ML que ya ha sido entrenado con un conjunto de datos grande y de propósito general y puede reutilizarse o ajustarse finamente para tareas específicas.
probabilistic	Comportamiento descrito en términos de probabilidades, donde los resultados son inciertos y se describen por probabilidades en lugar de certeza.

Término	Definición
random forest	Tecnología ML de conjunto para clasificación, regresión ML y otras tareas que opera construyendo y ejecutando muchos árboles de decisión y luego ya sea produciendo la moda de la clase o la predicción media de los árboles individuales.
recall	Una métrica de rendimiento funcional ML utilizada para evaluar un clasificador, que mide la proporción de positivos reales que fueron predichos correctamente. Sinónimo: sensibilidad (Según ISO/IEC TR 29119-11)
recurrent neural network	Un tipo de modelo de aprendizaje profundo diseñado para procesar datos secuenciales, permitiéndole reconocer patrones y dependencias a lo largo del tiempo.
reinforcement learning	Un enfoque en el cual un modelo ML, conocido como agente, aprende usando un bucle de retroalimentación de prueba y recompensa con su entorno para lograr objetivos específicos.
retrieval-augmented generation	Una técnica ML donde un sistema GenAI mejora dinámicamente su salida recuperando información externa relevante para complementar su conocimiento interno.
reward function	Una función que define el éxito del aprendizaje por refuerzo.
reward hacking	La actividad realizada por un agente inteligente para maximizar su función de recompensa en detrimento de cumplir el objetivo original. (Según ISO/IEC TR 29119-11)
search algorithm	Un algoritmo que visita sistemáticamente un subconjunto de todos los posibles estados o estructuras hasta que se alcanza el estado o estructura objetivo. (Según ISO/IEC TR 29119-11)

Término	Definición
self-learning system	Un sistema adaptativo que cambia su comportamiento basándose en el aprendizaje a través de prueba y error. (Según ISO/IEC TR 29119-11)
sensitive attributes	Características que definen grupos e individuos protegidos legal o éticamente que deben controlarse para prevenir la discriminación.
sentiment analysis	El uso de procesamiento de lenguaje natural y machine learning para identificar y clasificar el tono emocional expresado en texto.
stratified sampling	Una técnica para confirmar si una muestra representa proporcionalmente diferentes sub-poblaciones dentro de la población general.
super AI	Un tipo de IA que supera con creces las capacidades humanas.
supervised learning	Un enfoque para entrenar un modelo ML usando un conjunto de datos etiquetado.
support vector machine	Un algoritmo ML supervisado que encuentra el hiperplano óptimo entre puntos de datos para tareas de clasificación o regresión ML.
technological singularity	Un punto en el futuro cuando los avances tecnológicos ya no son controlables por las personas. (Según ISO/IEC TR 29119-11)
temperature	Una configuración que controla la aleatoriedad de las salidas de GenAI, con valores más bajos generando resultados más predecibles y valores más altos generando resultados más creativos.
test oracle problem	El desafío de determinar si una prueba ha pasado o fallado para un conjunto dado de entradas de prueba y estado.
training dataset	Un conjunto de datos utilizado para entrenar un modelo ML.

Término	Definición
transformer	Una arquitectura de red neuronal que procesa datos secuenciales para capturar dependencias de largo alcance, impulsando tareas de NLP, visión por computador y aplicaciones multimodales.
true negative	Una predicción en la cual el modelo predice correctamente la clase negativa.
true positive	Una predicción en la cual el modelo predice correctamente la clase positiva.
underfitting	La generación de un modelo ML que no refleja la tendencia subyacente del conjunto de datos de entrenamiento, resultando en un modelo que hace predicciones inexactas. (Según ISO/IEC TR 29119-11)
unsupervised learning	Un enfoque para entrenar un modelo ML usando un conjunto de datos no etiquetado.
validation dataset	Un conjunto de datos utilizado para evaluar un modelo ML entrenado con el propósito de ajustar el modelo.
von Neumann architecture	Una arquitectura de computador que consiste en cinco componentes principales: memoria, una unidad central de procesamiento, una unidad de control, entrada y salida.
weight	Una variable interna de una conexión entre neuronas en una red neuronal que afecta cómo computa sus salidas y que cambia a medida que la red neuronal se entrena.

10 Referencias

10.1 Estándares

- ISO/IEC/IEEE 12207 (2017), Systems and software engineering — Software life cycle processes
- ISO/IEC 2382 (2015), Information technology — Vocabulary
- ISO/IEC 22989 (2022), Information technology — Artificial intelligence — Artificial intelligence concepts and terminology
- ISO/IEC TR 24027 (2021) Information technology — Artificial intelligence (AI) — Bias in AI systems and AI aided decision making
- ISO/IEC 25010 (2023), Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model
- ISO/IEC 25059 (2023), Software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Quality model for AI systems
- ISO 26262-6 (2018), Road vehicles — Functional safety — Part 6: Product development at the software level
- ISO/IEC TR 29119-11 (2020), Software and systems engineering — Software testing — Part 11: Guidelines on the testing of AI-based systems
- ISO/IEC TS 42119-2 (2025) Artificial intelligence — Testing of AI – Part 2: Overview of testing AI systems
- ISO/IEC 42119 series (en desarrollo) Artificial intelligence — Testing of AI

10.2 Documentos ISTQB®

[CTFL] ISTQB® Certified Tester Foundation Level v4.0.1,
https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf (accedido 29.07.2025)

[CT-GenAI] ISTQB® Certified Tester – Testing with Generative AI (CT-GenAI), <https://istqb.org/certifications/gen-ai/> (accedido 19.12.2025)

10.3 Referencias del Glosario

Referencia para la terminología utilizada en este syllabus: - ISTQB® Glossary <https://glossary.istqb.org/>

10.4 Libros, Artículos y Páginas Web

[COV_REF] An Overview of Structural Coverage Metrics for Testing Neural Networks, Usman et al, Aug 2022, arXiv: 2208.03407 (accedido 29.07.2025)

[DATA_DOC] Gebru, T., Morgenstern, J., Vecchione, B., et al. (2021). Datasheets for Datasets. Communications of the ACM, 64(12), 86-92, <https://dl.acm.org/doi/pdf/10.1145/3502158> (accedido 07.10.2025)

[EU AI Act] EU Artificial Intelligence Act, July 2024, https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=OJ:L_202401689 (accedido 30.07.2025)

[MODEL_DOC] Mitchell, M., Wu, S., Varma, R., et al. (2019). Model Cards for Model Reporting. Proceedings of the Conference on Fairness, Accountability, and Transparency, <https://dl.acm.org/doi/10.1145/3287560.3287596> (accedido 07.10.2025)

[OECD AI] OECD Recommendation of the Council on Artificial Intelligence, May 2024, Organisation for Economic Co-operation and Development (OECD), <https://legalinstruments.oecd.org/en/instruments/oecd-legal-0449> (accedido 07.10.2025)

[UN Gov AI] Governing AI for Humanity: Final Report, September 2024, United Nations, https://www.un.org/sites/un2.un.org/files/governing_ai_for_humanity_final_report_en.pdf (accedido 29.07.2025)

[STATS] An Introduction to Statistical Learning: With Applications in R (2nd ed.) by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani, Springer.

11 Marcas Registradas

ISTQB® es una marca registrada de International Software Testing Qualifications Board.

12 Apéndice A - Objetivos de Aprendizaje/Nivel Cognitivo de Conocimiento

Los objetivos de aprendizaje específicos que aplican a este syllabus se muestran al inicio de cada capítulo. Cada tema en el syllabus será examinado según el objetivo de aprendizaje correspondiente.

Los objetivos de aprendizaje comienzan con un verbo de acción correspondiente a su nivel cognitivo de conocimiento como se enumera a continuación.

Nivel 1: Recordar (K1)

El candidato recordará, reconocerá y evocará un término o concepto.

Verbos de acción: Evocar, reconocer.

Ejemplos: - Evocar los conceptos de la pirámide de pruebas.
- Reconocer los objetivos típicos de las pruebas.

Nivel 2: Comprender (K2)

El candidato puede seleccionar las razones o explicaciones para declaraciones relacionadas con el tema, y puede resumir, comparar, clasificar y dar ejemplos del concepto de pruebas.

Verbos de acción: Clasificar, comparar, diferenciar, distinguir, explicar, dar ejemplos, interpretar, resumir

Ejemplos	Notas
Clasificar herramientas de prueba según su propósito y las actividades de prueba que soportan.	
Comparar los diferentes niveles de prueba.	Puede usarse para buscar similitudes, diferencias o ambas.
Diferenciar las pruebas de la depuración.	Busca diferencias entre conceptos.
Distinguir entre riesgos de proyecto y riesgos de producto.	Permite que dos (o más) conceptos se clasifiquen por separado.
Explicar el impacto del contexto en el proceso de prueba.	
Dar ejemplos de por qué las pruebas son necesarias.	
Inferir la causa raíz de defectos a partir de un perfil dado de fallas.	
Resumir las actividades del proceso de revisión de productos de trabajo.	

Nivel 3: Aplicar (K3)

El candidato puede llevar a cabo un procedimiento cuando se enfrenta a una tarea familiar, o seleccionar el procedimiento correcto y aplicarlo a un contexto dado.

Verbos de acción: Aplicar, implementar, preparar, usar

Ejemplos	Notas
Aplicar análisis de valores límite para derivar casos de prueba a partir de requisitos dados.	Debe referirse a un procedimiento/técnica/proceso, etc.

Ejemplos	Notas
Implementar métodos de recolección de métricas para apoyar requisitos técnicos y de gestión.	
Preparar pruebas de instalabilidad para aplicaciones móviles.	
Usar trazabilidad para monitorear el progreso de las pruebas para completitud y consistencia con los objetivos de prueba, la estrategia de prueba y el plan de prueba.	Podría usarse en un LO que quiere que el candidato pueda usar una técnica o procedimiento. Similar a 'aplicar'.

Nivel 4: Analizar (K4)

El candidato puede separar la información relacionada con un procedimiento o técnica en sus partes constituyentes para una mejor comprensión, y puede distinguir entre hechos e inferencias. La aplicación típica es analizar un documento, software o situación de proyecto y proponer acciones apropiadas para resolver un problema o tarea.

Verbos de acción: Analizar, deconstruir, delinear, priorizar, seleccionar.

Ejemplos	Notas
Analizar una situación de proyecto dada para determinar qué técnicas de prueba de caja negra o basadas en experiencia deben aplicarse para lograr objetivos específicos.	Examinable solo en combinación con un objetivo medible del análisis. Debe ser de la forma 'Analizar xxxx para xxxx' (o similar).
Priorizar casos de prueba en un conjunto de pruebas dado para ejecución basándose en los riesgos de producto relacionados.	
Seleccionar los niveles de prueba y tipos de prueba apropiados	Necesario donde la selección requiere análisis.

Ejemplos	Notas
para verificar un conjunto dado de requisitos.	

Referencia (Para los niveles cognitivos de los objetivos de aprendizaje) Anderson, L. W. and Krathwohl, D. R. (eds) (2001) A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives, Allyn & Bacon

13 Apéndice B - Matriz de Trazabilidad de Resultados de Negocio con Objetivos de Aprendizaje

Esta sección enumera la trazabilidad entre los Resultados de Negocio y los Objetivos de Aprendizaje del Certified Tester AI Testing.

La primera parte de la tabla muestra el número de Objetivos de Aprendizaje por Resultado de Negocio, mientras que la segunda parte muestra qué Objetivos de Aprendizaje están asociados con cada Resultado de Negocio.

Resultados de Negocio: AI Testing

ID	Resultado de Negocio	Número de LOs
BO1	Comprender el estado actual de la IA, incluyendo la IA generativa.	8
BO2	Experimentar la implementación y pruebas de modelos de machine learning.	10
BO3	Comprender el funcionamiento y las pruebas de redes neuronales simples.	2
BO4	Comprender las características de calidad específicas de IA definidas por ISO/IEC 25059.	3

ID	Resultado de Negocio	Número de LOs
BO5	Calcular e interpretar métricas de rendimiento funcional ML para modelos de machine learning.	1
BO6	Reconocer el alcance e importancia de los dos niveles de prueba específicos para las pruebas de sistemas de machine learning.	3
BO7	Contribuir al desarrollo de una estrategia de pruebas efectiva para un sistema de machine learning.	5
BO8	Diseñar y ejecutar casos de prueba para sistemas de machine learning.	14

Trazabilidad detallada de LOs por capítulo:

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO6
1 Introducción a la Inteligencia Artificial								
AI-1.1.1	Diferenciar entre sistemas basados en IA y sistemas convencionales	K2	X					
AI-1.1.2	Distinguir entre IA estrecha, IA general e IA superior	K2	X					
AI-1.1.3	Explicar los diferentes tipos de tecnologías de IA	K2	X					
AI-1.1.4	Explicar la IA generativa	K2	X					
AI-1.1.5	Comparar las opciones	K2	X					

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO6
	disponibles de hardware para implementar sistemas de machine learning							
AI-1.1.6	Comparar las opciones para el desarrollo y alojamiento de modelos de IA	K2	X					
AI-1.1.7	Resumir la funcionalidad proporcionada por los marcos de desarrollo ML	K2	X					
AI-1.1.8	Explicar cómo las regulaciones y estándares afectan el desarrollo y las pruebas de sistemas basados en IA	K2	X					
2 Características de Calidad para Sistemas Basados en IA								
AI-2.1.1	Clasificar comportamientos de sistemas basados en IA según las características de calidad definidas en ISO/IEC 25059	K2				X		
AI-2.1.2	Explicar las consideraciones especiales que	K2				X		

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO6
	surgen cuando se usa IA en sistemas relacionados con la seguridad							
AI-2.2.1	Dar ejemplos de criterios de aceptación para sistemas basados en IA	K2				X		
3 Machine Learning								
AI-3.1.1	Distinguir entre las diferentes formas de ML	K2		X				
AI-3.1.2	Resumir el flujo de trabajo utilizado para crear un sistema ML	K2		X				
AI-3.1.4	Resumir el uso de modelos preentrenados, ajuste fino y generación aumentada por recuperación	K2		X				
AI-3.2.1	Explicar las actividades relacionadas con la preparación de datos	K2		X				
AI-3.2.3	Contrastar el uso de conjuntos de datos de entrenamiento, validación y prueba en el	K2		X				

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO6
	desarrollo de un modelo ML							
AI-3.3.1	Calcular métricas comunes de rendimiento funcional ML a partir de un conjunto dado de datos de matriz de confusión	K3					X	
AI-3.4.1	Explicar la estructura y funcionamiento de una red neuronal profunda	K2			X			
AI-3.4.3	Describir las diferentes medidas de cobertura para redes neuronales	K2			X			
4 Pruebas de Sistemas Basados en IA								
AI-4.1.1	Comparar la testabilidad de sistemas basados en IA bloqueados y adaptativos	K2						
AI-4.1.2	Explicar por qué a menudo se necesita un enfoque estadístico al probar sistemas basados en IA	K2						
AI-4.1.3	Explicar los desafíos y	K2						

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO6
	soluciones relacionados con los oráculos de prueba para sistemas basados en IA							
AI-4.2.1	Explicar cómo se puede probar la IA generativa	K2						
AI-4.2.2	Implementar red teaming para sistemas GenAI	K3						
AI-4.3.1	Resumir los niveles de prueba utilizados para desarrollar sistemas de machine learning	K2						X
AI-4.3.2	Explicar cómo se aplican las pruebas basadas en riesgos a los sistemas de machine learning	K2						
5 Pruebas de Datos de Entrada para Sistemas de Machine Learning								
AI-5.1.1	Dar ejemplos de enfoques de prueba utilizados para la mitigación de riesgos de datos de entrada para un sistema de machine learning	K2						X

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO6
AI-5.1.2	Explicar cómo probar el sesgo	K2						
AI-5.1.3	Resumir las diversas formas de pruebas de pipeline de datos	K2						
AI-5.1.4	Explicar cómo probar la representatividad de los datos	K2						
AI-5.1.5	Aplicar pruebas de restricciones de conjuntos de datos	K3						
AI-5.1.6	Explicar las pruebas de corrección de etiquetas	K2						
6 Pruebas de Modelo para Sistemas de Machine Learning								
AI-6.1.1	Dar ejemplos de enfoques de prueba utilizados para la mitigación de riesgos de modelos ML	K2						X
AI-6.1.2	Explicar el propósito y enfoque de la revisión de documentación de modelos ML	K2						
AI-6.1.3	Explicar cómo se llevan a cabo las	K2						

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO6
	pruebas de rendimiento funcional ML para sistemas de machine learning probabilísticos							
AI-6.1.4	Resumir las pruebas adversariales de sistemas de machine learning	K2						
AI-6.1.5	Usar pruebas metamórficas para derivar casos de prueba para un escenario dado	K3						
AI-6.1.7	Explicar cómo se usan las pruebas de deriva en sistemas de machine learning operativos	K2						
AI-6.1.8	Explicar cómo se detectan el sobreajuste y el subajuste mediante pruebas	K2						
AI-6.1.9	Explicar cómo se usan las pruebas A/B en el contexto de sistemas de machine learning	K2						
AI-6.1.10	Explicar cómo se usan las pruebas back-to-back en el contexto de	K2						

LO	Objetivo de Aprendizaje	Nivel K	BO1	BO2	BO3	BO4	BO5	BO
	sistemas de machine learning							
7 Pruebas de Desarrollo de Machine Learning								
AI-7.1.1	Dar ejemplos de enfoques de prueba utilizados para la mitigación de riesgos del desarrollo ML	K2						X
AI-7.1.2	Explicar las diversas formas de pruebas de despliegue de sistemas ML	K2						

14 Apéndice C - Notas de Publicación

El ISTQB CT-AI v2.0 es una actualización y reescritura importante de la v1.0. Debido a la rápida evolución de la tecnología de IA, una actualización importante fue necesaria. El enfoque de la v2.0 está claramente en las pruebas de sistemas basados en IA. Debido a la publicación del ISTQB CT Testing with Generative AI, el capítulo sobre pruebas con IA fue completamente eliminado.

Esta versión principal ha realizado los siguientes cambios: - Introducción a la IA en general acortada - Inclusión de IA Generativa y pruebas de IA Generativa - Consolidación de las características de calidad de IA y sus desafíos - Enfoque reducido en métricas de rendimiento ML - Niveles de prueba refinados: Pruebas de datos de entrada y pruebas de modelo ML - Tipos de prueba refinados - Eliminación de pruebas con IA - Exclusión de entornos de prueba para pruebas de

sistemas basados en IA - Tiempo mínimo de formación requerido reducido de 4 días a 3 días

15 Índice

Todos los términos de pruebas están definidos en el Glosario ISTQB® (<https://glossary.istqb.org/>).

- A/B testing, 62
- accuracy, 35
- adaptive AI-based system (sistema basado en IA adaptativo), 41
- adversarial testing (pruebas adversariales), 59
- AI as a Service, 18
- AI functional correctness (corrección funcional de IA), 23
- AI robustness (robustez de IA), 23
- AI-based system (sistema basado en IA), 15, 40
- AI-based systems (sistemas basados en IA), 15
- API testing (pruebas de API), 67
- artificial intelligence (inteligencia artificial), 15
- association (asociación), 28
- attack (ataque), 43, 61
- back-to-back testing (pruebas back-to-back), 63
- canary testing (pruebas canary), 67
- classification (clasificación), 28, 34
- clustering (agrupamiento), 28
- concept drift (deriva de concepto), 61
- confusion matrix (matriz de confusión), 35
- data drift (deriva de datos), 61
- data pipeline testing (pruebas de pipeline de datos), 50
- data preparation (preparación de datos), 32
- data representativeness testing (pruebas de representatividad de datos), 51

- dataset constraint testing (pruebas de restricciones de conjuntos de datos), 52
- device compatibility testing (pruebas de compatibilidad de dispositivos), 67
- disparate impact analysis (análisis de impacto dispar), 50
- drift testing (pruebas de deriva), 61
- dynamic testing (pruebas dinámicas), 48, 50
- EU AI Act, 20
- explainability (explicabilidad), 24
- exploratory data analysis (análisis exploratorio de datos), 29, 33, 49
- exploratory testing (pruebas exploratorias), 44
- F1-score, 35
- fine-tuning (ajuste fino), 31
- follow-up test case (caso de prueba de seguimiento), 60
- frontier AI, 15
- functional adaptability (adaptabilidad funcional), 23, 25
- functional correctness (corrección funcional), 22, 25
- general AI (IA general), 16
- generative AI (IA generativa), 17, 43
- input data testing (pruebas de datos de entrada), 45, 48
- installability testing (pruebas de instalabilidad), 67
- intervenability (intervenibilidad), 23, 26
- k-multisection neuron coverage (cobertura neuronal de k-multisección), 38, 68
- label correctness testing (pruebas de corrección de etiquetas), 50, 53
- large language model (modelo de lenguaje grande), 32, 42
- locked AI-based system (sistema basado en IA bloqueado), 40
- machine learning, 15, 16, 28
- metamorphic relation (relación metamórfica), 60

- metamorphic testing (pruebas metamórficas), 60, 61
- ML algorithm (algoritmo ML), 29
- ML development framework (marco de desarrollo ML), 19, 29
- ML development testing (pruebas de desarrollo ML), 65
- ML functional performance (rendimiento funcional ML), 58, 65
- ML functional performance criteria (criterios de rendimiento funcional ML), 30
- ML functional performance metric (métrica de rendimiento funcional ML), 34
- ML functional performance metrics (métricas de rendimiento funcional ML), 29
- ML model (modelo ML), 18, 29, 31, 33
- ML model documentation (documentación de modelo ML), 57
- ML model testing (pruebas de modelo ML), 56
- ML regression (regresión ML), 28
- ML workflow (flujo de trabajo ML), 29, 30, 34
- model conversion testing (pruebas de conversión de modelo), 67
- model testing (pruebas de modelo), 45
- multiple annotation (anotación múltiple), 53
- narrow AI (IA estrecha), 15
- neural network (red neuronal), 36, 37
- neuron boundary coverage (cobertura de frontera neuronal), 38, 69
- neuron coverage (cobertura neuronal), 38
- non-determinism (no determinismo), 24
- non-functional test (prueba no funcional), 30, 60
- overfitting (sobreajuste), 62
- perceptron (perceptrón), 38
- precision (precisión), 35
- pretrained model (modelo preentrenado), 31
- recall, 35
- red teaming, 43

- reinforcement learning (aprendizaje por refuerzo), 16, 28
- retrieval augmented generation (generación aumentada por recuperación), 32
- review (revisión), 49, 57
- risk-based testing (pruebas basadas en riesgos), 46
- robustness (robustez), 26
- rollback testing (pruebas de reversión), 67
- safety (seguridad funcional), 24, 26
- self-learning (autoaprendizaje), 24
- shadow testing, 67
- societal and ethical risk mitigation (mitigación de riesgos sociales y éticos), 22, 23, 26
- source test case (caso de prueba fuente), 60
- static analysis (análisis estático), 49
- super AI (IA superior), 16
- supervised learning (aprendizaje supervisado), 16, 28
- test dataset (conjunto de datos de prueba), 30, 34
- test oracle (oráculo de prueba), 41, 60
- testing for bias (pruebas de sesgo), 49
- training dataset (conjunto de datos de entrenamiento), 33
- transparency (transparencia), 23, 24, 25
- underfitting (subajuste), 62
- unsupervised learning (aprendizaje no supervisado), 16, 28
- user controllability (controlabilidad del usuario), 23, 25
- validation dataset (conjunto de datos de validación), 29, 33